



UNIVERSITÀ DEGLI STUDI ROMA TRE

Department of Civil, Computer Science and
Aeronautical Technologies Engineering

Master's Degree Course in Computer Engineering

Simultaneous Embedding with Fixed Edges of Graphs with Biconnected Intersection

By

Giordano Andreola

Supervisor

Prof. Giordano Da Lozzo

Academic Year 2023/2024

A chi mi ha sempre sopportato.

Acknowledgments

Prima di tutto ci tengo ad iniziare con il ringraziare di cuore il professor Giordano Da Lozzo, per avermi guidato in questo affascinante mondo del disegno di grafi, e per avermi assiduamente (molto assiduamente, le voglio bene ♡) seguito durante lo svolgimento di questo lavoro di tesi. È stato di grande aiuto.

Un ringraziamento speciale va alla mia famiglia, in primo luogo ringrazio i miei genitori che, nonostante raramente sentono pronunciare queste parole (o in generale, qualsiasi altra parola), sanno che gli voglio bene. Anche i restanti membri della mia famiglia si meritano uno speciale ringraziamento, ma sono troppi per essere elencati tutti. Per questo motivo li elencherò tutti ugualmente. Ringrazio le nonne Nonna Laura e Nonna Vanna, gli zii Zio Stefano e Zia Cristina, Zio Mauro e Zia Marina, Zio Pino e Zia Teresa. Ringrazio anche le mie cugine più grandi Arianna e Laura, e i miei cuginetti più piccoli Mattia e Riccardo. Per me ci siete tutti sempre stati, e so che ci sarete sempre.

Ringrazio il mio gruppo di amici del liceo (più un imbucato), amici che ormai, considerando da quanto ci conosciamo, posso iniziare a ritenere d'infanzia. Un gruppo di disagiati al punto da farsi chiamare "i gravi". In ordine di capelli rimasti sono: Marins, Alessio, Bianco, Cristo e Gab. Spero che le disavventure con voi non finiranno mai. Ringrazio anche altri tre membri non ufficiali del gruppo, Simone, Dionino ed Elena.

Ringrazio Mirko, un caro amico che (purtroppo) conosco da quando ho memoria. Per me rimarrà sempre un mistero come tu faccia ad 1) essere così scarso a biliardino 2) essere così brutto.

Ringrazio i miei amici dell'università, i quali hanno contribuito ad alleggerire le mie giornate a lezione e appesantire le mie vacanze estive. In particolare sono Valerio, detto Er Chiacchera (oppure Er sta sessione do fisica), la strega antipatica Beatrice, Alberello,

universalmente noto come il dio delle challenge, Gianluca (quello secco), Ferneti (quello ancora più secco), Cardilli (lo so che nella tua lista speciale avrò sempre un posto riservato), la dolce donzella Valentina, che ormai ha l'abbonamento dei passaggi in macchina (basta che io continuo ad avere l'abbonamento delle torte fatte in casa), Alexis, el dio del balo e dei siti web, il Carello, quello scarso a calcio ma forte a tressette, il Tocu (ovvero papà orso), Andrea, il dio dei progetti di infovis, e per ultime, ma non per importanza, le dolci donzelle dee dell'arrivare in ritardo agli appuntamenti Mary ed Elisa.

Ringrazio anche altri amici a cui voglio bene, nonostante in questo ultimo periodo li ho visto poco: Simone, Claudia ed Alessandro.

Ringrazio infine i miei compagni di basket, giocare a questo sport con voi mi rende il quotidiano più leggero. Tra tutti dedico un ringraziamento speciale a Leo, Lollo, e Riccardo (ma sì, uno anche a Luca).

Contents

Contents	v
List of Figures	vii
List of Algorithms	xi
Abstract	xiv
Introduction	xv
1 Basic Definitions	1
2 The Auslander-Parter Algorithm	4
2.1 Theoretical Background	4
2.2 The Testing Algorithm	8
2.2.1 Computing Segments of a Cycle	10
2.2.2 Non-separating Cycles	11
2.2.3 The Interlacement Graph	15
3 An Algorithm for SEFE with Biconnected Intersection	18
3.1 Theoretical Background	18
3.2 The Testing Algorithm	28
3.2.1 Computing Union Segments	29
3.2.2 Non-separating Cycles	30
3.2.3 The SEFE Interlacement Graph	33
3.3 The Embedding Algorithm	36

4 The Web Tool	51
4.1 WebAssembly	51
4.2 Description of the Web Tool	52
4.3 The Web Tool Interface	52
Conclusions and Future Developments	55
Bibliography	57

List of Figures

1.1	A planar drawing of a planar graph. Each biconnected component in the graph is depicted with a different color, and cut-vertices are the only named nodes.	2
1.2	Figures a), b) and c) represent three planar graphs, whose pairwise intersection coincides (sunflower SEFE). In d) we have a drawing representing an admissible SEFE of the three graphs. The intersection is drawn with black edges, while edges strictly belonging to one of the three graphs have the color of the graph they come from. Note that there is a crossing between a green and a blue edges, i.e. a crossing between an edge of b) and an edge of c).	3
2.1	A cycle in a graph (depicted with thick black edges) and its segments, S_0 , S_1 and S_2 depicted in red, blue and green respectively. Attachments are duplicated in order to be represented within the segments.	6
2.2	Figure a) shows a pair of segments in conflict, while figure b) shows a pair of segments not in conflict.	6
2.3	Figure a) depicts a cycle in black, and its four segments, S_0 , S_1 , S_2 and S_3 , which are drawn respectively in green, purple, red and blue. Figure b) shows the resulting interlacement graph of these segments. Note that such interlacement graph is bipartite.	7
2.4	The two base cases of the algorithm: a) A chord and b) a path.	10
2.5	Figure a) shows a non separating cycle, and its only non-base case segment. Figure b) shows a separating one, with the two resulting segments.	12

2.6	Figure a) depicts two consecutive attachments a_0 and a_1 of a non-base case segment (note that they have a non-attachment cycle node in between, however they are still considered to be consecutive). In this case the path of the cycle that will get substituted, in order to compute a separating cycle, is the Red one, since the Blue one contains other attachment nodes, a_2 and a_3 . Figure b) shows an example of two paths in the cycle that both do not contain other attachment nodes.	12
3.1	Planar graphs G_0 (a) and G_1 (b) and their union graph G_U (c).	19
3.2	Two example of union segments of a union graph. The one on the left has a segment S_{U0} and a chord S_{U1} , while the one on the right has only a segment. The attachment a_0 of S_0 is Blue-Red , while a_1 is Black . The attachments of S_{U1} are both Blue	20
3.3	Figure a) depicts a Black cycle, and its four union segments, S_{U0} , S_{U1} , S_{U2} and S_{U3} . Figure b) shows the resulting interlacement graph of these segments. Note that such interlacement graph is bipartite.	23
3.4	In figure a) is depicted a Black cycle C_B that does not separate, and its only non-base case union segment S_U . In this case there exist two consecutive Black attachment that do not have any other attachment in between, a_0 and a_1 . The path between them α that does not contain other attachment is substituted, in C_B , by the path γ connecting them in S_U . This way α becomes a new union segment S_{U0} of C'_B , and we have another union segment S_{U1} , as shown in figure b).	25
3.5	Figure a) depicts the case of a Black cycle C_B with a non base case union segment S_U such that there are no consecutive Black attachments in C_B without other attachments in between. From figure b) we can clearly see that any path in S_U connecting w and w' contains a node of γ' . In figure c) we can see that if this is not the case, there has to exist an edge η (of any color) that crosses with γ'	25

- 3.6 In this examples the cycle's starting point, and its traversing order is indicated by the black arrow. The segment's in a) **Red** min attachment is a_1 , while its **Red** max attachment is a_0 , with respective values of 1 and 7. Its **Blue** min attachment is also a_1 , while its **Blue** max attachment is instead a_3 , with respective values of 1 and 5. The segment has both a middle **Red** attachment and a middle **Blue** attachment. The segment's in b) **Red** min attachment is a_1 , as well as its **Blue** min attachment, with a value of 3. Its **Red** max attachment is a_0 , as well as its **Blue** max attachment, with a value of 7. The segment does not have neither a middle **Red** attachment nor a middle **Blue** attachment. 27
- 3.7 This figure represent the same segments from Fig. 3.6 in such a way to make more understandable the concept of min and max attachments, as well as middle attachments. Basically, regarding one of the two projections of the union segment, the min and max attachment of this projection are the most "far away" placed attachments in C_B , the leftmost and the rightmost ones. Regarding the **Blue** portion of the union segment in a), a_2 places itself between its **Blue** min and max attachments, which are respectively a_1 and a_3 . That's why the segment has a **Blue** middle attachment. Same thing applies for its **Red** projection. 27
- 3.8 Figure a) shows a non-separating cycle with a non-base case union segment $S_{\cup 0}$. Note that in any path in the cycle between the two black attachments there is another non-**Black** attachment. Figure b) shows the new separating **Black** cycle and its union segments $S_{\cup 0}$ (which is a **Blue** chord) and $S_{\cup 1}$. . . 31
- 3.9 The two drawings come from the same simultaneous embedding. We can see in figure a), where the cycle is drawn clockwise, that its union segment is drawn inside the cycle. In figure b) the cycle is instead drawn counter clockwise, and the same union segment is placed outside of the cycle. . . . 37
- 3.10 This figure shows the union segments adjacent to a cycle node v . In this case, the clockwise encountering order of segments, regarding v , is S_1 , S_0 and then S_2 40

3.11	In this SEFE instance, with reference to the node v , belonging to both segments S_{U0} and S_{U1} , we must place the segments in a specific clockwise order around v in order to avoid crossings. Specifically, we need to place S_{U0} after S_{U1} , as shown in figure a). If we do not follow this ordering, such as in figure b), there will be crossings.	45
3.12	The union segments in both figures share the same Red min and max attachments. In a) S_{U0} has a Red middle attachment, while in b) S_{U0} does not. Note that in figure a), the Blue projection of the union segments does not induce an ordering, since S_{U0} does not have a Blue middle attachment, it results in a draw, like the one depicted in b).	46
3.13	In this SEFE instance, with reference to the node v , belonging to both segments S_{U0} and S_{U1} , we must place S_{U0} after S_{U1} in both figures a) and b) in order to avoid crossings. In figure a) v is a Red min attachment of S_{U0} and a Red max attachment of S_{U1} . In figure a) v keeps being a Red min attachment of S_{U0} and it is a Red middle attachment of S_{U1}	46
3.14	In this SEFE instance, with reference to the node v , which is a Red max attachment of both union segments S_{U0} and S_{U1} in both figures a) and b), we must place S_{U0} before S_{U1} in both figures a) and b) in order to avoid crossings. In figure a) S_{U0} has a Red min attachment with a lower value of the one of S_{U1} . In figure b) S_{U0} has a Red middle attachment.	47
3.15	In this SEFE instance, with reference to the node v , which is a Red max attachment of S_{U0} in both figures a) and b), we must place S_{U0} before S_{U1} in both figures a) and b) in order to avoid crossings. In figure a) v is a Red min attachment of S_{U1} , while in figure b) it is a Red middle attachment. . .	48
3.16	Figure a) depicts orderings of the two attachments such that the union segments do not cross with each other, while figure b) represent the opposite case in which they do.	48
4.1	The figure shows a screenshot of the Auslander and Parter page in our web tool.	53
4.2	The figure shows a screenshot of the SEFE page in our web tool.	54

List of Algorithms

1	Function that tests planarity of a Biconnected Graph.	9
2	Function that computes the segments of a cycle in a biconnected graph.	10
3	Helper function for ComputeSegments that populates a segment using a depth first search, and marks attachment nodes.	11
4	Function to compute a separating cycle from a non-separating one.	13
5	Function to compute a path in a cycle between two attachments of a segment. The path does not contain any other attachment node of the segment.	13
6	Helper function for ComputePathInCycle . The function tries to find a path in the cycle between two attachments n_0 and n_1 , following u , one of the two neighbors of n_0 . If following the node u leads to a path with another attachment, an empty list is otherwise returned.	14
7	Function that changes a path inside a cycle with another path.	15
8	Function to compute the interlacement graph from a cycle and its segments.	16
9	Function to check if a pair of Segments is in conflict.	16
10	Function that labels the nodes of a cycle for efficient alternating attachments searching.	17
11	Function to check whether an union graphs with biconnected intersection admits a SEFE.	28
12	Function to compute the union segments of a Black cycle in a union graph.	29
13	Helper function for ComputeUnionSegments to populate a union segment using a depth first search. It marks attachment nodes of the segment, and label them with their color.	30

14	Function to compute a separating Black cycle from a non-separating one.	32
15	Function to compute a path in a Black cycle between two attachments.	32
16	Helper function for <code>ComputePathInCycleSefe</code> . The function tries to find a path in the Black cycle between two Black attachments n_0 and n_1 , following u , one of the two neighbors of n_0 . If following the node u leads to a path with another Black attachment, an empty list is otherwise returned.	33
17	Function to compute the Sefe interlacement graph from the Black cycle and the union segments	34
18	Function to check if a pair of union segments is in conflict.	35
20	Function to project a union segment to its Blue component.	35
19	Function to project a union segment to its Red component.	35
21	Function to compute a simultaneous embedding for a positive instance of SEFE $G_\cup$ with $G_\cap$ biconnected.	38
22	Function to compute a simultaneous embedding for an instance of SEFE $G_\cup = S_\cup \cup C_B$, with $S_\cup$ being a base case segment, i.e. a path or a chord, and C_B being the Black cycle originating it. If C_B is drawn clockwise, $S_\cup$ gets placed inside of C_B	39
23	Function to compute SEFE for an instance of SEFE $G_\cup$ given a Black cycle C_B of $G_\cup$, and a list of SEFEs Γ_i containing a SEFE of $S_{\cup i} \cup C_B$ for each union segment of C_B	41
24	Given a cycle, assuming it is drawn clockwise, this function determines if a segment of the cycle gets placed inside the cycle, given the embedding of their union.	42
25	Function that add to the output embedding the edges of a Black cycle node, that belongs to a union segment.	43
26	Function that add to the output embedding the edges of non-attachment nodes that belongs to a union segment.	43

27	Assumes the cycle is drawn clockwise. Given the position pos of a node in the cycle, this function determines the clockwise order of segments, that share the cycle node in pos , assuming they all need to be placed outside of the cycle, such that they do not create intersections.	44
28	Given two union segments of a Black cycle, both sharing an attachment v , this function returns an index which establishes, assuming these segments need to be placed outside the cycle, the clockwise order around v that they have to be placed in, such that crossings are avoided.	49
29	Given two union segments that share the same two attachments, this function establishes an ordering between such segments around the cycle node in position pos	50

Abstract

Graphs are among the most versatile and expressive objects in the field of Computer science, but they are also among the most complex. Graph drawing is a research area at the intersection of Computer science and Mathematics that studies algorithms, models, and techniques for the effective visualization of graphs. A graph is planar if it can be drawn on a plane without intersecting edges. In a simultaneous embedding with fixed edges of multiple planar graphs, such graphs are visualized at the same time in such a way that the vertices shared among them have the same position and that no crossing occurs between edges belonging to the same graph. This reduces the visual complexity of the representation and facilitates the exploration of the relational information embedded in the graphs. The goal of this thesis is to design and implement an algorithm for the problem of testing for the existence of simultaneous embedding with fixed edges of graphs (for short, the SEFE Problem), and compute one such drawing if any. In particular, we devised a novel algorithm for the SEFE Problem of pairs graphs whose intersection is biconnected. The algorithm is inspired by the famous algorithm conceived by Auslander and Parter for testing the planarity of graphs and retains the same time complexity. Moreover, the algorithm has been implemented as a C++ library adopted in a WebAssembly application, enabling an efficient, web-based interface that allows the visualization of the generated layouts.

Keywords: graph, graph drawing, planarity, simultaneous embedding

Introduction

Information Visualization is the practice of representing data in a meaningful, visual way such that users can easily reason over it, making the comprehension and interpretation simpler. Of all the data structure that can be visualized, one attracts the Computer Science community in a more prominent way: Graphs, which are structures made of vertices and edges, where the edges represent relations between pairs of vertices. In order to visualize graphs in the most meaningful way, there exist well-established criteria to be met, the most basic ones being: The length of all the drawn edges has to be as equal as possible, and there have to be as few crossings between edges as possible. Many algorithms to visualize graphs with these features in mind exist, for example Force-directed algorithms assign forces between nodes and then compute the nodes' position such that the total energy is minimized.

As we have just seen, one of the criteria aims to minimize crossings between edges in the drawing. Actually, there are graphs that can always be drawn with no crossings at all; such graphs are called planar. The study on planar graphs is a classic subject, comprising a vast body of deep theoretical results. Many algorithms to establish whether a graph is planar in linear time exist. The first planarity testing algorithm to run in polynomial time was developed by Auslander and Parter [AP61] in 1961, even though in its original form it contained an error that allowed it to loop indefinitely in some cases. This error was later fixed by Goldstein [Gol63] in 1963. In the first part this work, we will dive deep into the planarity testing algorithm by Auslander and Parter, providing all the theoretical background needed and the procedures pseudocode. The main focus of this thesis, however, is about another problem regarding graph planarity in the context of multiple graphs: The Simultaneous Embedding with Fixed Edges

(SEFE) Problem. A simultaneous embedding with fixed edges consists of visualizing more graphs at the same time, in a way such that all common vertices of the graphs have the same position in the drawing, and no edges belonging to the same graph cross with each other. It has been proved that the SEFE problem for three or more graphs is NP-complete [ADN15]. Determining the computational complexity of the SEFE problem for two graphs remains one of the major open challenges in the field of Graph Drawing. In this work, we will analyze a restricted SEFE problem, that is, verifying whether two graphs with biconnected intersection admit a simultaneous embedding with fixed edges and, in case they do, compute it. Even though we are only trying to simultaneously embed two graphs, note that, under the biconnected intersection restriction, these ideas apply for arbitrarily-many graphs.

The algorithm we propose for this restricted SEFE problem takes strong inspiration from the Auslander and Parter algorithm, and a result obtained by Bläsius, Karrer, and Rutter [BKR15]. It retains Auslander and Parter running time of $O(|V|^2)$, where V is the set of nodes of the two graphs. Note that, however, in current literature there are algorithms capable of determining whether two graphs with biconnected intersection admit a simultaneous embedding with fixed edges in linear time [ADF⁺12], but their approach is fairly complex compared to the one we propose. Our algorithm's simplicity is what actually allowed it to be smoothly implemented.

In this work, we implemented both the Auslander and Parter and the newly-devised SEFE algorithm, using a low level programming language: C++. We then compiled the code into WebAssembly, using the Emscripten compiler, to create a web tool that makes the implemented library accessible to anyone using a web browser. The visualization of the planar embedding, in case of positive instances, is made with the C++ library OGDF [CGJ⁺23]. The Auslander and Parter algorithm implementation had the only role of laying the foundations for the library that implements our SEFE algorithm.

This thesis is organized with the following structure:

- in Chapter 1, we will provide preliminary concepts and basic definitions needed in order to understand the subsequent chapters;
- in Chapter 2, we will discuss the theoretical background of the Auslander and Parter algorithm and show the planarity testing algorithm's procedures pseudocode;

- in Chapter 3, we will dive deep into the SEFE problem of two graphs with the biconnected intersection, describing our novel testing and embedding algorithm, together with all the needed pseudocode;
- in Chapter 4, we will give a brief presentation of the developed web tool containing all the implemented algorithms seen in this work; and
- in the last Chapter, we will provide concluding remarks and discuss possible future developments of this thesis.

Chapter 1

Basic Definitions

In this chapter we will provide basic definitions and preliminaries concepts for the understanding of the following work.

Graph Theory. A *unordered graph* $G = (V, E)$ is a structure made of an ordered pair: V , a set of vertices and $E \subseteq \{\{u, v\} \mid u, v \in V \wedge u \neq v\}$, a set of edges with each edge represented as an unordered pair of vertices $\{u, v\}$. From now on, we will refer to unordered graphs simply as *graphs*. A *path* in $G = (V, E)$ is a sequence of vertices $v_0, v_1, \dots, v_{k-1}$ with $v_i \in V$ for every i such that there is an edge $e \in E$ between each consecutive pair of vertices (v_i, v_{i+1}) in the sequence, and $v_i \neq v_j$ for $i \neq j$. A *cycle* in G is a path $v_0, v_1, \dots, v_{k-1}$ with $k \geq 3$ of G such that there is an edge between v_0 and v_{k-1} . A *chord* of G given a cycle C is an edge in G between two non-consecutive vertices in C . A graph G is *connected* if every pair of vertices is connected by a path. A *k-connected* graph G is such that removing any $k - 1$ vertices leaves G connected; 3-connected and 2-connected graphs are also called *triconnected* and *biconnected*, respectively. A *cut-vertex* is a vertex whose removal disconnects the graph. A *separation pair* is a pair of vertices whose removal disconnects the graph. A *tree* is a connected graph with no cycle. The *degree* of a vertex is the number of vertices adjacent to it. A graph $G = (V, E)$ is *bipartite* if the set of vertices V can be divided into two disjoint sets of vertices U and W such that every edge $e = (u, v) \in E$ has $u \in U$ and $v \in W$. A *depth first search* is an algorithm used for traversing or searching through graph data structures $G = (V, E)$. It explores as far down a branch as possible before backtracking,

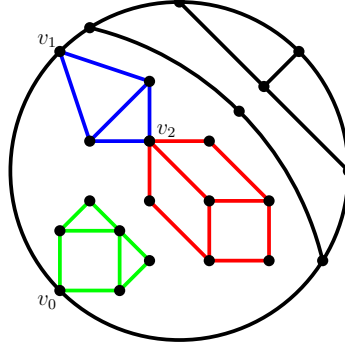


Figure 1.1: A planar drawing of a planar graph. Each biconnected component in the graph is depicted with a different color, and cut-vertices are the only named nodes.

and its time complexity is $O(|V| + |E|)$.

A *breadth first search* is an algorithm used for traversing or searching through graph data structures $G = (V, E)$. It explores all the vertices at the present depth level before moving on to vertices at the next depth level, and its time complexity is $O(|V| + |E|)$.

Graph Drawing. A *drawing* of a graph is a mapping of each vertex to a point of the plane and of each edge to a simple curve connecting its endpoints. A drawing is *planar* if the curves representing its edges do not intersect except, possibly, at common endpoints. A graph is *planar* if it admits a planar drawing. A planar drawing Γ determines a subdivision of the plane into connected regions, called *faces*, and a clockwise ordering of the edges incident to each vertex, called *rotation scheme*. The unique unbounded face is the *outer face*. Two drawings of the same connected graph are equivalent if they have the same rotation schemes. We define an *embedding* Γ of $G = (V, E)$ as a data structure associating each node v in V with an adjacency list containing a permutation of the k neighbor nodes of v , representing the clockwise ordering of the edges incident to each vertex, i.e. the rotation scheme. Note that, in G , edges in E are represented as unordered pairs of vertices (u, v) , meaning that $(u, v) = (v, u)$. This way we do not need to duplicate each edge in order to represent both possibilities, which makes sense for unordered graphs. However, in an embedding Γ of G , in order to represent the rotation schemes of all the nodes, each node u in v 's adjacency list also has v in its adjacency list, basically representing twice each edge. An embedding is *planar* if its resulting drawing is planar. A planar embedding is an equivalence class of planar drawings.

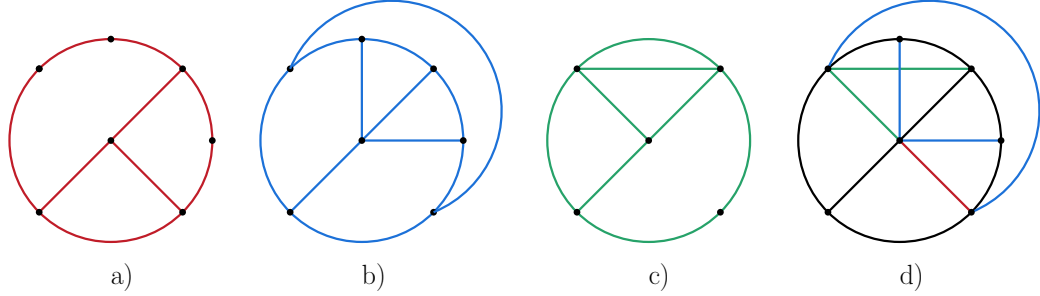


Figure 1.2: Figures a), b) and c) represent three planar graphs, whose pairwise intersection coincides (sunflower SEFE). In d) we have a drawing representing an admissible SEFE of the three graphs. The intersection is drawn with black edges, while edges strictly belonging to one of the three graphs have the color of the graph they come from. Note that there is a crossing between a green and a blue edges, i.e. a crossing between an edge of b) and an edge of c).

SEFE. Let $G_0, \dots, G_{k-1}$ be k graphs on the same set V of vertices. A *simultaneous embedding with fixed edges* (SEFE) of $G_0, \dots, G_{k-1}$ consists of k planar drawings $\Gamma_0, \dots, \Gamma_{k-1}$ of $G_0, \dots, G_{k-1}$, respectively, such that each vertex $v \in V$ is mapped to the same point in every drawing Γ_i and each edge that is common to more than one graph is represented by the same Jordan arc in the drawings of all such graphs. The *SEFE problem* is the problem of testing whether k input graphs $G_0, \dots, G_{k-1}$ admit a SEFE. When the intersection $G_i \cap G_j$ is the same for all $0 \leq i, j < k$, the problem is defined as *sunflower SEFE*. It has been proved that the SEFE problem regarding three or more graphs is NP-complete, even under the sunflower SEFE restriction [ADN15]. The focus of this work is on instances of Sunflower SEFE with exactly two graphs. In particular, we aim at testing the existence of a SEFE of two graphs (that is, when $k = 1$) with biconnected intersection. In this case, the problem can be studied both in terms of embeddings and in terms of drawings, since edges can be represented by arbitrary curves without geometric restrictions, and since Jünger and Schulz [JS09] proved that two graphs G_0 and G_1 with intersection graph $G_\cap$ have a SEFE if and only if there exists a planar embedding Γ_0 of G_0 and a planar embedding Γ_1 of G_1 inducing the same embedding of $G_\cap$. In fact, our algorithm for testing the SEFE problem extends to the case of arbitrary k .

Chapter 2

The Auslander-Parter Algorithm

In 1961, Auslander and Parter published the first algorithm capable of deciding the planarity of a graph $G = (V, E)$ in polynomial time¹ $O(|V|^2)$. However, the original algorithm they proposed in [AP61] contained an error that allowed it to loop indefinitely. This error was later corrected by Goldstein in his work [Gol63].

The algorithm proposed by Auslander and Parter is based on the presence of cycles in a biconnected graph. Once a cycle is identified, the removal of such cycle from the graph yields the components, called "segments", that will be the main figures of the algorithm's subsequent recursive calls.

2.1 Theoretical Background

In this chapter, we will dive into the theoretical concepts needed to understand the algorithm proposed by Auslander And Parter.

Property 1. *A graph G is planar if and only if its biconnected components are planar.*

The biconnected components of a graph $G = (V, E)$ can be efficiently computed in $O(|V| + |E|)$, using the algorithm developed by Hopcroft and Tarjan reported in [HT73], based on a depth first search of the graph.

¹Although the original paper by Auslander and Parter claims an $O(|V|^3)$ running time, a tight analysis of the algorithm shows an actual $O(|V|^2)$ time complexity.

Property 2. *A biconnected graph G with three or more vertices contains (at least) a cycle.*

Theorem 1 (Euler). *For each planar connected graph $G = (V, E)$ the following equality holds: $|V| - |E| + |F| = 2$, where $|F|$ denotes the number of faces induced by the embedding of G .*

Corollary 1. *For each planar connected graph $G = (V, E)$ with $|E| \geq 2$ the number of edges is limited by $|E| \leq 3 \cdot |V| - 6$.*

Proof. Let f_i denote the number of faces that are incident to i sides of edges. Since $|E| \geq 2$, each face is incident to at least three sides, so the total number of faces is given by $|F| = \sum_{i=3}^{2 \cdot |E|} f_i$. Every edge has exactly two sides, so it follows that

$$2 \cdot |E| = \sum_{i=3}^{2 \cdot |E|} i \cdot f_i \geq \sum_{i=3}^{2 \cdot |E|} 3f_i \implies |F| \leq \frac{2}{3} \cdot |E|$$

By applying Theorem 1 we find that $2 = |V| - |E| + |F| \leq |V| - \frac{1}{3} \cdot |E|$, hence $|E| \leq 3 \cdot |V| - 6$. $\square$

Corollary 1 implies that we will never have to actually run the planarity testing algorithm on graphs with $|E|$ not linear in $|V|$.

Given a graph $G = (V, E)$, let C be a cycle in G . A *segment* S of C in G is either a connected component in $G \setminus C$, together with the vertices and edges connecting it to C , or a chord of C . The vertices that a segment S and the cycle C share are called the attachments of S .

A cycle is called *separating* if its segments are at least two.

Theorem 2 (Jordan Curve Theorem [Jor87]). *Every simple closed curve in the plane separates its complement into two connected, non-empty sets: the interior region and exterior region.*

Corollary 2. *No two points from the exterior and interior region of a simple closed curve can be connected by a line that does not intersect with the curve.*

Let G be a connected graph and C a cycle in G . If G admits a planar representation Γ , C must be a closed curve in Γ . Corollary 2 affirms that any segment S_i of C must be either placed inside the cycle, or entirely outside of it.

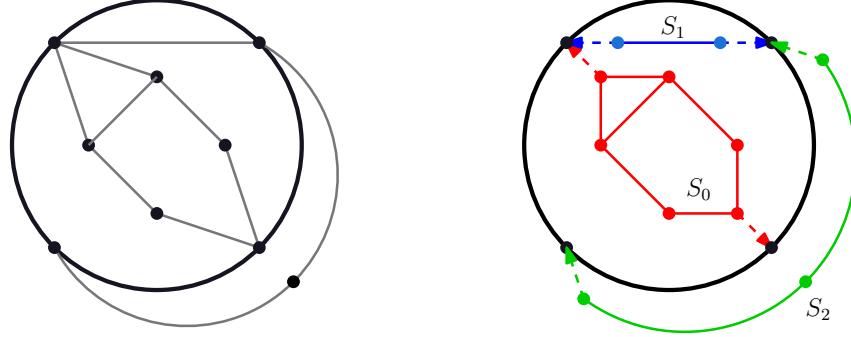


Figure 2.1: A cycle in a graph (depicted with thick black edges) and its segments, S_0 , S_1 and S_2 depicted in red, blue and green respectively. Attachments are duplicated in order to be represented within the segments.

Given a graph G and a cycle C , two segments S_i and S_j of the cycle C with attachments $a_0, \dots, a_{n-1}$ and $b_0, \dots, b_{m-1}$ respectively, are in *conflict* if and only if, while traversing the cycle, four attachments appear in the following order: $a_i, b_j, a_{i'}, b_{j'}$. Fig. 2.2 shows an example of segments in conflict.

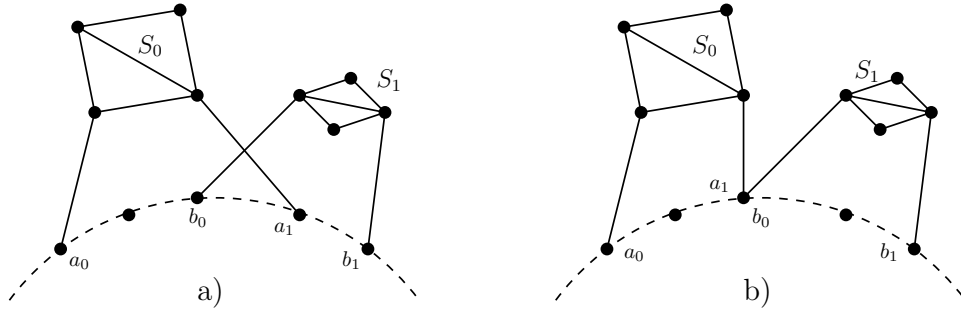


Figure 2.2: Figure a) shows a pair of segments in conflict, while figure b) shows a pair of segments not in conflict.

Theorem 3 (Theorem 1 from [AP61]). *Let G be a planar graph and let C be any cycle in G . Then, G admits a planar representation where C is adjacent to the external face if and only if no two segments S_i and S_j of C are in conflict.*

Theorem 4 (Theorem 4 from [AP61]). *Let G be a connected graph. Let C be any cycle in G . Then G is planar if and only if:*

- all the segments of C can be partitioned in two sets such that no two elements from the same set are in conflict;
- for each segment S_i , the graph $G_i = S_i \cup C$ is planar.

Theorem 4 lays the foundations for the recursive algorithm to test if a graph G is planar. We learn from it that, in order for G to admit a planar embedding, it is necessary that the segments of the cycle C in G can be partitioned in two sets such that no two segments from the same set are in conflict. This way one set gets embedded outside the cycle, and the other set gets embedded inside of it.

The *interlacement graph* $I_{G,C}$ induced by G and a cycle C in G , is a graph in which each node represents a segment S_i , and two segments in conflict are linked by an edge; see Fig. 2.3

Using this definition we can reformulate Theorem 4 in a more convenient way as such:

Theorem 5. *Let G be a connected graph. Let C be any cycle in G . Then G is planar if and only if:*

- the interlacement graph $I_{G,C}$ is bipartite;

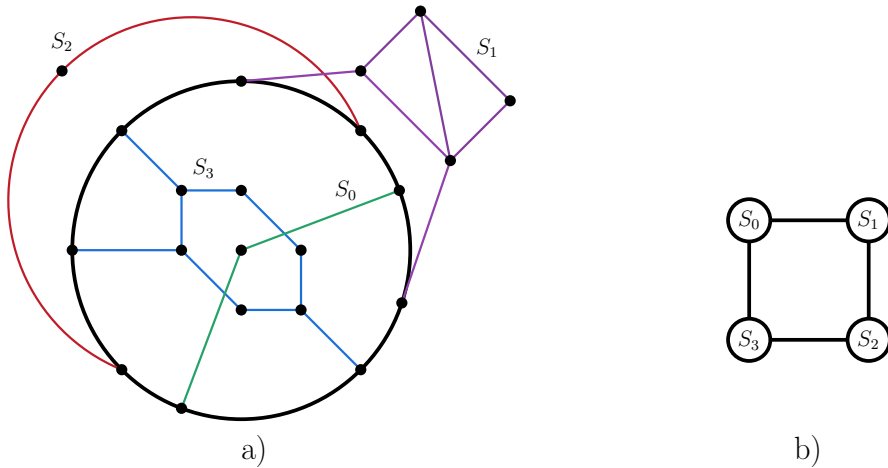


Figure 2.3: Figure a) depicts a cycle in black, and its four segments, S_0 , S_1 , S_2 and S_3 , which are drawn respectively in green, purple, red and blue. Figure b) shows the resulting interlacement graph of these segments. Note that such interlacement graph is bipartite.

- for each segment S_i , the graph $G_i = S_i \cup C$ is planar.

Lemma 1 ([BETT99]). *Let G be a biconnected graph, let C be a non-separating cycle of G , and let S be its only segment. If S is not a path, then G has a separating cycle C' consisting of a subpath of C plus a path γ of S between two attachments.*

Proof. Let u and v be two attachments of S that are consecutive in the circular ordering of C , let α be a subpath of C between u and v that does not contain any other attachment of S , and let β be the subpath of C between u and v different from α . Since S is connected, there is a path γ in S between u and v . Let C' be the cycle obtained from C by replacing α with γ . We have that α is a segment of G with respect to C' . If S is not a path, let η be an edge of S not in γ . There is a segment of C' distinct from α containing η . Therefore, if S is not a path, then C' has at least two segments and is thus a separating cycle of G . $\square$

Lemma 1 affirms that we can always keep finding separating cycles in $S_i \cup C$, otherwise the segment is one of the two base cases: a chord or a path. The lemma's proof provides the procedure to compute a separating cycle from a non-separating one.

2.2 The Testing Algorithm

Theorem 5 lays the foundations for a recursive algorithm, whose pseudocode is given in Algorithm 1. The input is going to be any biconnected graph G , while the output will be a boolean value, which establishes whether G is planar. We will assume that the input graph G is neither a cycle nor just two nodes connected by an edge, both these cases are trivially planar. This assumption guarantees that, when a cycle C is identified, the number of resulting segments is at least one.

The first step of the algorithm involves identifying a cycle C , whose segments S_i for $0 \leq i < n$ will produce the components $G_i = S_i \cup C$ for the following recursive calls.

In the algorithm pseudocode are present procedures whose pseudocode, when the procedure is not trivial, will be given in the following sections. In particular `ComputeSegments` will be discussed in Section 2.2.1, `AdjustCycle` in Section 2.2.2, and `ComputeInterlacementGraph` in Section 2.2.3. Trivial functions like `ComputeCycle` and

`IsGraphBipartite`, will not be discussed in a chapter of their own. `ComputeCycle` computes any cycle C given a graph G , which can be easily be done with a depth first search or a broad first search in G . `IsGraphBipartite` verifies for the graph given in input to be bipartite, which can also be easily done with a depth first search of a broad first search.

The intuition behind Algorithm 1 is that, in a planar embedding of a graph, after identifying a cycle C in a biconnected graph G , any resulting segment can either be entirely placed inside the cycle or entirely outside of it; otherwise, it would create crossings with the cycle; see Corollary 2.

Two segments S_0 and S_1 placed in the same side of the cycle C , however, may intersect with each other. Auslander and Parter, in their work, gave an easy characterization of conflicts between a pair of segments, solely based on the positions of their attachment nodes. Specifically, recall that two segments are in conflict if and only if their attachments alternate, as shown in Fig. 2.2.

Algorithm 1 Function that tests planarity of a Biconnected Graph.

Input: a biconnected graph G

Output: *true* if G is planar, *false* otherwise

```

1: procedure TESTPLANARITY( $G$ )
2:   if  $|E| > 3 \cdot |V| - 6$  then
3:     return false
4:    $C \leftarrow \text{COMPUTECYCLE}(G)$  ▷ computes a cycle in  $G$ 
5:    $segments \leftarrow \text{COMPUTESEGMENTS}(G, C)$ 
6:   if  $|segments| == 1$  then
7:     if  $S$  in  $segments$  is a path then ▷ the base case
8:       return true
9:     else ▷ chosen cycle is not separating
10:       $C \leftarrow \text{ADJUSTCYCLE}(S, C)$ 
11:       $segments \leftarrow \text{COMPUTESEGMENTS}(G, C)$ 
12:    $I_{G,C} \leftarrow \text{COMPUTEINTERLACEMENTGRAPH}(C, segments)$ 
13:   if not ISGRAPHBIPARTITE( $I_{G,C}$ ) then
14:     return false
15:   for  $S$  in  $segments$  do
16:      $G' \leftarrow S \cup C$ 
17:     if not TESTPLANARITY( $G'$ ) then
18:       return false
19:   return true

```

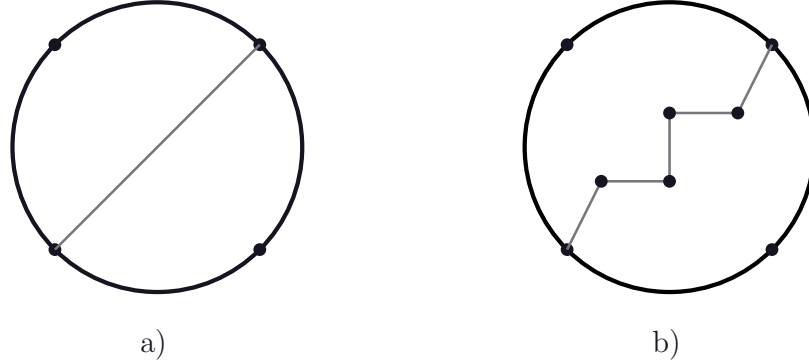


Figure 2.4: The two base cases of the algorithm: a) A chord and b) a path.

Algorithm 2 Function that computes the segments of a cycle in a biconnected graph.

Input: a biconnected graph G , a cycle C

Output: the segments of C in G

```

1: procedure COMPUTESEGMENTS( $G, C$ )
2:    $segments \leftarrow$  empty list
3:   for  $v$  in  $C$  do
4:     mark  $v$  as visited
5:   for  $v$  in  $V$  do
6:     if  $v$  is not visited then
7:        $S \leftarrow$  empty segment ▷ i.e. empty graph
8:       POPULATESEGMENTDFS( $G, C, S, v$ )
9:       add  $S$  to  $segments$ 
10:  return  $segments$ 

```

The algorithm will keep recurring until it reaches one of the two base cases, which are trivially planar: a chord or a path; see Fig. 2.4.

2.2.1 Computing Segments of a Cycle

Given a biconnected graph G and a cycle C , computing the cycle's segments can be done by executing slightly modified visits of G : nodes will be expanded in the visit unless they are already visited, which means they belong to the cycle or to other segments. All found nodes this way will belong to the same segment S . Algorithm 2 provides a pseudocode for the algorithm that computes segments of a cycle C in a biconnected graph G .

Algorithm 2 utilizes a helper function to populate a segment whose pseudocode is in

Algorithm 3 Helper function for `ComputeSegments` that populates a segment using a depth first search, and marks attachment nodes.

Input: a biconnected graph G , a cycle C , a segment S , a node v

```

1: procedure POPULATESEGMENTDFS( $G, C, S, v$ )
2:   add  $v$  to  $S$ 
3:   mark  $v$  as visited
4:   for  $u$  in  $v$  neighbors do
5:     if  $(u, v)$  is not in  $S$  edges then
6:       add  $(u, v)$  to  $S$  edges
7:     if  $u$  is in  $C$  and  $u$  is not in  $S$  nodes then
8:       add  $u$  to  $S$  nodes
9:       mark  $u$  as attachment
10:    if  $u$  is not visited then
11:      POPULATESEGMENTDFS( $G, C, S, u$ )

```

Algorithm 3. The helper function recursively keeps expanding nodes with a depth first search as long as the found nodes are not already visited (note that cycle's nodes are already set as visited). This way, all the found nodes will belong to the same segment. The function will also label attachment nodes as *attachment*.

2.2.2 Non-separating Cycles

What Auslander and Parter missed in their work is that the algorithm they proposed may loop indefinitely. An unfortunate choice of the cycle C in G may result in the cycle not being *separating*. We will have only one instance to launch the next recursive call, which coincides exactly with the original graph G . If the same cycle keeps being chosen, then no progress will be made. This error was fixed by Goldstein [Gol63] some year after the publication of the original paper by Auslander and Parter.

If the choice of the cycle C results in only one segment, which is not one of the two base cases, we have to recompute the cycle in order to make it *separating*.

This can always be done, as we have seen in Lemma 1, by substituting a path, in the cycle, that links two consecutive attachment nodes, with any path inside the segment that also links the two attachments; see Fig. 2.5.

When considering a path between two attachments in the cycle, there will be two possible ones; the one we have to substitute is the one not containing any other attachment in between (if the segment only has two attachments, both the paths do not

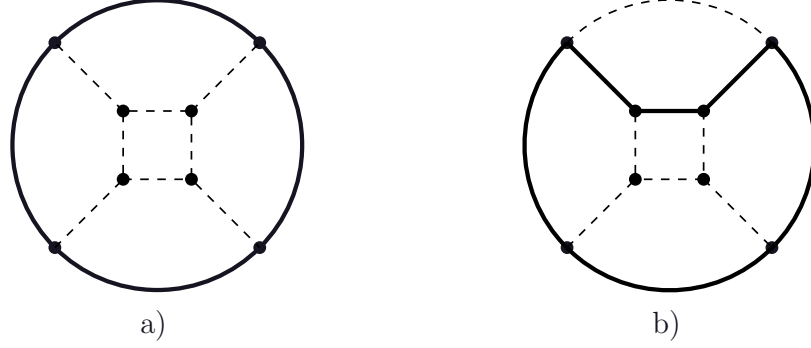


Figure 2.5: Figure a) shows a non separating cycle, and its only non-base case segment. Figure b) shows a separating one, with the two resulting segments.

contain any other attachment in between, in this case the choice is arbitrary). See the example given in Fig. 2.6.

It is worth noting that, since the instances are biconnected, there will always be such path inside the segment between two attachments. For the same reason, any segment will always contain at least two attachments.

This way the algorithm can keep recurring until it actually reaches one of the trivially planar base cases. Since we can always find separating cycle in non-base cases, the following sub-instances $G_i = S_i \cup C$ are strictly smaller than G , guaranteeing that the

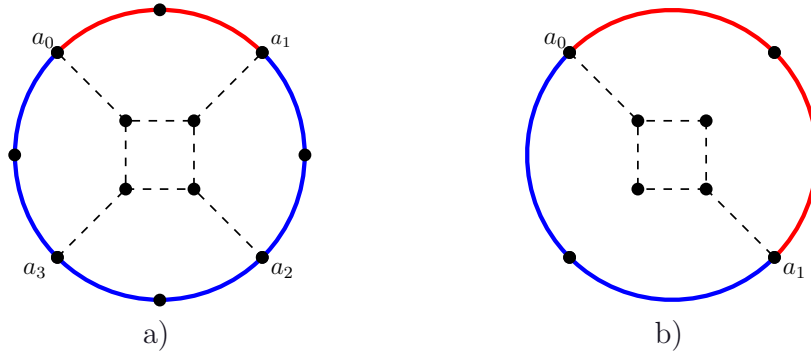


Figure 2.6: Figure a) depicts two consecutive attachments a_0 and a_1 of a non-base case segment (note that they have a non-attachment cycle node in between, however they are still considered to be consecutive). In this case the path of the cycle that will get substituted, in order to compute a separating cycle, is the Red one, since the Blue one contains other attachment nodes, a_2 and a_3 . Figure b) shows an example of two paths in the cycle that both do not contain other attachment nodes.

Algorithm 4 Function to compute a separating cycle from a non-separating one.

Input: a segment S , the cycle C

Output: a new *separating* cycle C

```

1: procedure ADJUSTCYCLE( $S, C$ )
2:    $attachments \leftarrow$  empty list
3:   for  $v$  in  $C$  do                                      $\triangleright$  traversing the cycle in clockwise order
4:     if  $v$  is attachment of  $S$  then
5:       add  $v$  to  $attachments$ 
6:     if  $|attachments| == 2$  then
7:       break                                            $\triangleright$  list now contains two consecutive attachments
8:    $a_0 \leftarrow attachments[0]$ 
9:    $a_1 \leftarrow attachments[1]$ 
10:   $cyclePath \leftarrow$  COMPUTEPATHINCYCLE( $C, a_0, a_1, S$ )
11:   $segmentPath \leftarrow$  COMPUTEPATH( $S, a_0, a_1$ )
12:                                      $\triangleright$  Computes a path between two nodes in a graph
13:  return CHANGEPATH( $C, cyclePath, segmentPath$ )

```

algorithm cannot loop indefinitely.

Algorithm 4 provides the algorithm pseudocode to compute and return a separating cycle, given in input a non separating cycle C , and the non-base case resulting segment S of C . Firstly, two **consecutive** (in C) attachments nodes a_0 and a_1 will be found, then the helper function **ComputePathInCycle**, whose pseudocode is provided in Algorithm 5, computes the path in C , between those two attachments, that does not contain any other attachment node of S , as explained before.

This path in C will be then substituted, by the helper function **ChangePath**, described in Algorithm 7, with any path in the segment S that connects the two attachments a_0

Algorithm 5 Function to compute a path in a cycle between two attachments of a segment. The path does not contain any other attachment node of the segment.

Input: a cycle C , an attachment a_0 , an attachment a_1 , a segment S

Output: a list of nodes representing a path

```

1: procedure COMPUTEPATHINCYCLE( $C, a_0, a_1, S$ )
2:    $u_0 \leftarrow$  first neighbor of  $a_0$ 
3:    $u_1 \leftarrow$  second neighbor of  $a_0$ 
4:    $path \leftarrow$  FOLLOWNODE( $C, a_0, a_1, S, u_0$ )
5:   if  $path$  is empty then
6:      $path \leftarrow$  FOLLOWNODE( $C, a_0, a_1, S, u_1$ )
7:   return  $path$ 

```

Algorithm 6 Helper function for `ComputePathInCycle`. The function tries to find a path in the cycle between two attachments n_0 and n_1 , following u , one of the two neighbors of n_0 . If following the node u leads to a path with another attachment, an empty list is otherwise returned.

Input: a cycle C , a node n_0 , a node n_1 , a segment S , a node u

Output: a list of nodes representing a path, or an empty list

```

1: procedure FOLLOWNODE( $C, n_0, n_1, S, u$ )
2:    $path \leftarrow$  empty list
3:   add  $n_0$  to  $path$ 
4:    $prev \leftarrow n_0$ 
5:   while true do
6:     add  $u$  to  $path$ 
7:     if  $u = n_1$  then
8:       return  $path$ 
9:     if  $u$  is an attachment in  $S$  then
10:      return empty list      ▷ We do not want other attachments in the path
11:     $temp \leftarrow u$ 
12:    if  $u$ 's first neighbor =  $prev$  then
13:       $u \leftarrow u$ 's second neighbor
14:    else
15:       $u \leftarrow u$ 's first neighbor
16:     $prev \leftarrow temp$ 

```

and a_1 . This last path gets computed by the helper function `ComputePath`, that simply computes any path in the input graph between any two nodes, assuming it exists.

The function `ComputePathInCycle` needs to find a path in a cycle C connecting two attachments that does not contain any other attachment of S . In order to do so, as we can see in its provided pseudocode in Algorithm 5, it utilizes an helper function `FollowNode`. This function, described in Algorithm 6, computes a path from a_0 to a_1 following one of the two neighbors of a_0 . If following one neighbor results in a path containing other attachment nodes, the other neighbor will be used. Since the two attachments a_0 and a_1 are consecutive in C , at least one of those two paths must not contain other attachments of S .

Finally, function `ChangePath`, whose pseudocode is shown in Algorithm 7, has the job of substituting the old path in C with the new one found in the segment S . Clearly, the old path and the new path must have the same endpoints. After adding all the nodes of the new path to the new cycle, it also adds the nodes and links in C that do

Algorithm 7 Function that changes a path inside a cycle with another path.

Input: a segment S , the cycle C

Output: a new *separating* cycle C

```

1: procedure CHANGEPATH( $C, oldPath, newPath$ )
2:    $C' \leftarrow$  empty graph
3:   for  $0 \leq i < |newPath| - 1$  do
4:     add  $newPath[i]$  to  $C'$  nodes
5:     add  $(newPath[i], newPath[i + 1])$  to  $C'$  edges
6:   add last  $newPath$  element to  $C'$  nodes
7:   for  $v$  in  $C$  do
8:     if  $v$  is in  $oldPath$  then
9:       continue
10:    add  $v$  to  $C'$ 
11:    for  $u$  in  $v$  neighbors do
12:      if  $(u, v)$  not in  $C'$  edges then
13:        add  $(u, v)$  to  $C'$  edges
14:   return  $C'$ 

```

not belong to the path that gets removed from C .

2.2.3 The Interlacement Graph

As stated in Theorem 4, in order to check for the planarity of the input biconnected graph G , we need to check whether the set of the segments of C can be partitioned in two sets such that no two segments from the same set are in conflict. To ease the process, we defined the interlacement graph $I_{G,C}$ such that we only have to check for it being bipartite, as stated in Theorem 5, which can be done with a broad first search of $I_{G,C}$.

Algorithm 8 provides the pseudocode for the function **ComputeInterlacementGraph**, which is responsible for creating the interlacement graph $I_{G,C}$ of G given a cycle C and its segments. It utilizes the function **AreSegmentsInConflict**, whose pseudocode is provided in Algorithm 9, in order to establish the pairs of conflicting segments.

Two segments S_i and S_j of a cycle C are in conflict if they cannot be placed on the same side of C without creating intersections between edges. For instance, conflict can be established by verifying that a pair of attachments from S_i alternates with a pair of attachments from S_j . Algorithm 9 provides an efficient way of determining alternating attachments. It is described in Zeijlemaker Sjanne Bachelor's thesis work [Zei].

Algorithm 8 Function to compute the interlacement graph from a cycle and its segments.

Input: a cycle C , the segments induced by C

Output: the Interlacement Graph $I_{G,C}$

```

1: procedure COMPUTEINTERLACEMENTGRAPH( $C, segments$ )
2:    $I_{G,C} \leftarrow$  empty graph
3:   for  $S$  in  $segments$  do
4:     add a node to  $I_{G,C}$ 
5:   for  $0 \leq i < |segments| - 1$  do
6:     for  $i < j < |segments|$  do
7:       if ARESEGMENTSINCONFLICT( $C, segments[i], segments[j]$ ) then
8:         add  $(i, j)$  to  $I_{G,C}$  edges
9:   return  $I_{G,C}$ 

```

Algorithm 9 Function to check if a pair of Segments is in conflict.

Input: a cycle C , a segment S_0 , a segment S_1

Output: *true* if the segments are in conflict, *false* otherwise

```

1: procedure ARESEGMENTSINCONFLICT( $C, S_0, S_1$ )
2:   COMPUTECYCLELABELS( $C, S_0$ )
3:    $n \leftarrow$  |attachments of  $S_0$ |
4:    $labels \leftarrow$  array of size  $2 \cdot n$ 
5:   initialize  $labels$  with 0's
6:    $sum \leftarrow 0$ 
7:   for  $v$  attachment of  $S_1$  do
8:      $labels[\text{label of } v] \leftarrow 1$   $\triangleright$  label of  $v$  assigned in ComputeCycleLabels
9:   for  $0 \leq i < 2 \cdot n$  do
10:     $sum \leftarrow sum + labels[i]$ 
11:    $partSum \leftarrow labels[0] + labels[1] + labels[2]$ 
12:   for  $0 \leq i < n$  do
13:     if  $partSum = sum$  then
14:       return false
15:      $partSum \leftarrow partSum + labels[(3 + 2i) \bmod 2k] + labels[(4 + 2i) \bmod 2k]$ 
16:      $partSum \leftarrow partSum - labels[2i] - labels[(1 + 2i) \bmod 2k]$ 
17:   return true

```

Function `ComputeCycleLabels`, described in Algorithm 10 labels the nodes of the cycle, using a segment S as such:

- attachments $a_0, \dots, a_{k-1}$ of S are labeled with $0, 2, \dots, 2(k-1)$. The indexes denote the order in which the attachments are encountered while traversing the cycle.

- Let $v \in C$ be a cycle vertex that is not an attachment of S . If v lies between a_j and a_{j+1} , v is labelled with $2j + 1$.

Algorithm 10 Function that labels the nodes of a cycle for efficient alternating attachments searching.

Input: a cycle C , a segment S

```

1: procedure COMPUTECYCLELABELS( $C, S$ )
2:    $totalAttachments \leftarrow |\text{attachments of } S|$ 
3:    $label \leftarrow 0$ 
4:   for  $v$  in  $C$  do                                      $\triangleright$  traversing the cycle in order
5:     if  $v$  is an attachment of  $S$  then
6:       assign  $(2 \cdot label)$  to  $v$ 
7:        $label \leftarrow label + 1$ 
8:     else
9:       if  $label = 0$  then
10:        assign  $(2 \cdot totalAttachments - 1)$  to  $v$ 
11:       else
12:        assign  $(2 \cdot label - 1)$  to  $v$ 

```

Chapter 3

An Algorithm for SEFE with Biconnected Intersection

In this chapter, we will illustrate an approach to the Simultaneous Embedding with Fixed Edges problem that takes strong inspiration from the algorithm proposed by Auslander and Parter and a work of Bläsius, Karrer and Rutter [BKR15]. In particular, the proposed algorithm solves SEFE for graphs with biconnected intersection in polynomial time. In particular, given two planar graphs $G_0 = (V, E_0)$ and $G_1 = (V, E_1)$, the algorithm's running time complexity is $O(|V|^2)$, retaining Auslander and Parter algorithm's running time complexity. To be precise, the time complexity is actually $O(|E_0 \cup E_1|^2)$, but since the two input graphs G_0 and G_1 are planar, their respective number of edges is linearly bounded by the number of nodes (see Corollary 1), implying that $|E_0 \cup E_1|$ also has to be linearly bounded by $|V|$.

3.1 Theoretical Background

We define a special graph suited for the simultaneous embedding problem. Let $G_0 = (V, E_0)$ and $G_1 = (V, E_1)$ be two planar graphs, the *union graph*

$$G_{\cup} = (V, E_0 \cup E_1, \phi: E_0 \cup E_1 \longrightarrow \{\text{Blue}, \text{Red}, \text{Black}\})$$

is an ordered triple with the third element being a function associating each edge with a label out of three available:

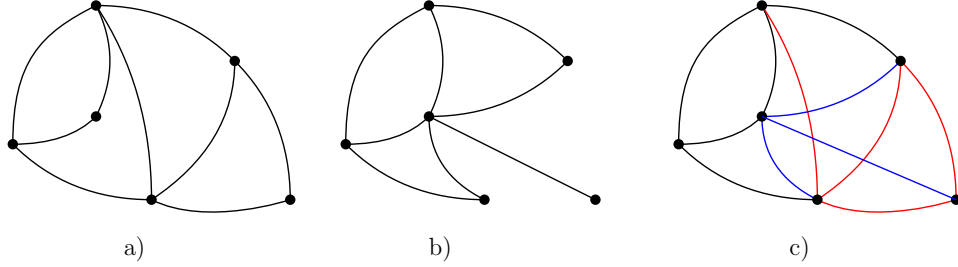


Figure 3.1: Planar graphs G_0 (a) and G_1 (b) and their union graph G_U (c).

- **Blue:** if $(e \in E_0) \wedge (e \notin E_1)$,
- **Red:** if $(e \notin E_0) \wedge (e \in E_1)$,
- **Black:** if $e \in (E_0 \cap E_1)$.

The **Blue** and **Red** edges refer to edges belonging to only one of the two graphs, while the **Black** ones to edges present in both graphs (see Fig. 3.1).

In the remainder of the section, we also denote an instance (G_0, G_1) of SEFE by means of the union graph G_U of (G_1, G_2) . This allows us to simplify the notation, by using a single graph instead of two different ones. Recall that, in order for an instance $G_U = (V, E, \phi)$ to admit a SEFE, only intersections between **Red** and **Blue** edges are allowed.

In the following, consider an instance $G_U = (V, E, \phi)$ of SEFE. We define the *intersection of G_U* as $G_\cap = (V, e \in E \text{ if } \phi(e) = \text{Black})$. We define a *Black cycle* of G_U as a cycle in $G_\cap$ (i.e. a cycle C_B of **Black** edges in G_U). Given a **Black** cycle C_B , a *union segment* S_U of C_B on G_U is either a connected component in $G_U \setminus C_B$ together with the vertices and edges connecting it to C_B , or a *colored chord* of C_B : a chord of C_B labeled with the edge's color. The vertices that a union segment S_U and cycle C_B have in common are called the *attachments* of S_U . Each attachment v in S_U also has labels based on his incident edges in the segment (see Fig. 3.2):

- **S_U -Blue:** if only **Blue** edges in S_U are incident to v ,
- **S_U -Red:** if only **Red** edges in S_U are incident to v ,
- **S_U -Blue-Red:** if only **Blue and Red** edges in S_U are incident to v ,

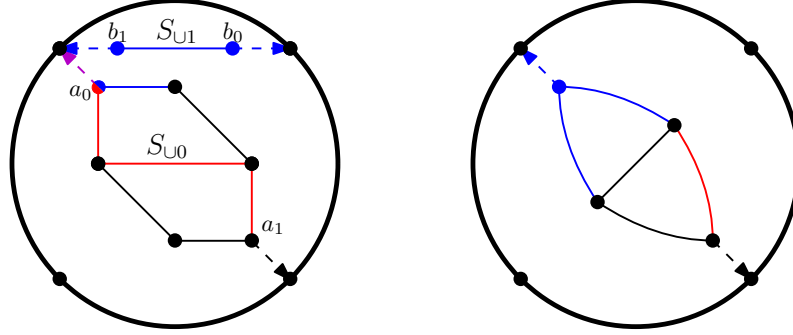


Figure 3.2: Two example of union segments of a union graph. The one on the left has a segment S_{U0} and a chord S_{U1} , while the one on the right has only a segment. The attachment a_0 of S_0 is Blue-Red, while a_1 is Black. The attachments of S_{U1} are both Blue.

- S_U -Black: if any Black edge in S_U is incident.

We define the following subgraphs of G_U . The *red projection* G_{Red} of G_U is the subgraph of G_U induced by its Red and Black edges, that is,

$$G_{Red} = (V, e \in E \text{ if } \phi(e) = \text{Red} \vee \phi(e) = \text{Black}).$$

Analogously, the *blue projection* G_{Blue} of G_U is the subgraph of G_U induced by its Blue and Black edges, that is,

$$G_{Blue} = (V, e \in E \text{ if } \phi(e) = \text{Blue} \vee \phi(e) = \text{Black}).$$

We have the following.

Lemma 2. *Let G_U be an instance of SEFE whose intersection $G_\cap$ is biconnected, and let C_B be a cycle in $G_\cap$ with union segments $S_{U0}, S_{U1}, \dots, S_{Uk-1}$. For every $0 \leq i < k$, let $G_\cap^i$ be $(C_B \cup S_{Ui})$, then $G_\cap^i$ is biconnected.*

Proof. Let us assume $G_\cap^i$ is not biconnected for $0 \leq i < k$. Then it must have at least one cutvertex. Since C_B is made of only Black edges, C_B is entirely contained in $G_\cap$. Clearly, summing all the intersections of union segments $S_{U0}, S_{U1}, \dots, S_{Uk-1}$ together with the cycle C_B results in $G_\cap$. By definition, any pair of union segments S_{Ui} and S_{Uj} with $i \neq j$ cannot have any common node, outside of attachment nodes. This means that the cutvertex in the intersection of $G_\cap^i$ would remain as such even after summing all

the other segments, since there cannot be any back edges from any node of the segment to any other node not belonging to the segment, which would result in an articulation point in $G_\cap$. $\square$

Since the above lemma implies that an instance $G_\cup$ of SEFE can be split into sub-instances $G_\cup^i$ induced by the segment $S_{\cup 1}$, for $i = 0, \dots, k - 1$, whose intersection graph $G_\cap^i$ is biconnected, it paves the way for a recursive algorithm to test the SEFE problem for instances whose intersection is biconnected.

We define *colored paths* in $G_\cup$ as such:

- *Black-path*: a path in $G_\cup$ made of only **Black** edges,
- *Red-path*: a path in $G_\cup$ made of only **Black** and **Red** edges,
- *Blue-path*: a path in $G_\cup$ made of only **Black** and **Blue** edges.

The next lemma shows the existence of certain monochromatic paths between attachments.

Lemma 3. *Let $G_\cup = (V, E, \phi)$ be an instance of SEFE, with $G_\cap$ biconnected. Let C_B be a cycle in $G_\cap$ with its union segments $S_{\cup i}$ for $0 \leq i < n$. Then:*

- *each **Blue** attachment in $S_{\cup i}$ will have a **Blue** path connecting it to every other **Blue**, **Black**, and **Blue-Red** attachment in $S_{\cup i}$, and vice-versa,*
- *each **Red** attachment in $S_{\cup i}$ will have a **Red** path connecting it to every other **Red**, **Black**, and **Blue-Red** attachment in $S_{\cup i}$, and vice-versa,*
- *each **Black** attachment in $S_{\cup i}$ will have a **Black** path connecting it to every other **Black** attachment in $S_{\cup i}$.*
- *no same-colored path connects a **Red** attachment in $S_{\cup i}$ with a **Blue** attachment in $S_{\cup i}$.*

Proof. If the union segment $S_\cup$ is a chord, by definition its only edge determines the color of both $S_\cup$'s two attachments, which are trivially connected by the edge. If the segment $S_\cup$ is not a chord, it has at least one non-attachment node. Let $G_\cup^*$ be

$(C_B \cup S_U)$, we know by Lemma 2 that $G_\cap^*$ is biconnected. This means that between every non-attachment node in S_U there is a **Black** path connecting them (since **Black** edges also count as both **Blue** and **Red**, there is also always a **Blue** or **Red** path connecting them). By definition, each attachment has at least one edge of its color linking it to a non attachment node, so between two attachments of the same color there will always be a path of that color connecting them. **Blue** attachments will also have **Blue** path connecting them to **Blue-Red** attachments; **Red** attachments will have **Red** path connecting them to **Blue-Red** attachments. A **Red** attachment in S_U , by definition, only has **Red** edges connecting it to other nodes in S_U , so no **Blue** path can connect it to any node. Likewise, a **Blue** attachment in S_U only has **Blue** edges connecting it to other nodes in S_U , so no **Red** path can connect it to any node. $\square$

The following lemma, due to Bläsius, Kurrer, and Rutter, provides the counterpart of Theorem 3 in the context of SEFE.

Lemma 4 (Lemma 7 from [BKR15]). *Let $G_U = (V, E, \phi)$ an instance of SEFE, and let C_B be a cycle in $G_\cap$, with union segments $S_{U0}, S_{U1}, \dots, S_{Uk-1}$. Then G_U admits a SEFE where each union segment is embedded on the same side of C_B if and only if (i) each union segment admits a SEFE together with C_B and (ii) no two union segments of C_B have projections of the same color with alternating attachments.*

Lemma 4 sets up the way for the following theorem.

Theorem 6. *Let $G_U = (V, E, \phi)$ an instance of SEFE, and let $G_\cap$ be its intersection. If there is a cycle C_B in $G_\cap$, then G_U admits a SEFE if and only if the following conditions hold:*

- *For each union segment S_U of C_B , the union graph $G'_U = C_B \cup S_U$ admits a SEFE.*
- *The interlacement graph of the union segments of C_B is bipartite. Two union segments are considered in conflict if they have projections of the same color with alternating attachments.*

Proof. Suppose first that G_U admits a SEFE Γ . Each subgraph obtained by merging a union segment S_U with C_B also admits a SEFE. Note that, in a topological drawing

of G_U in which a Black cycle C_B is drawn without self intersections, such a cycle is represented as a simple closed curve in the plane, it follows that each union segment of C_B must be drawn either completely in the interior or completely outside the exterior of the region delimited by C_B in Γ , otherwise by Theorem 2 there would be crossings, since there cannot be crossings between Black edges and any other edge. This partitions the union segments into two distinct sets by Lemma 4, so the interlacement graph of the union segments is bipartite.

Suppose the interlacement graph of the segments of C_B is bipartite and the graph $C_B \cup S$ admits a Sefe for each union segment S_U . Then the union segments can be partitioned into two sets such that each set contains no pair of conflicting union segments. From Lemma 4, it follows that all union segments in each set can be drawn on the same side of C_B without compromising the SEFE. Assign one set to the exterior region and one to the interior region of C_B to obtain a planar drawing of G_U . $\square$

This theorem, clearly strongly inspired by its counterpart Theorem 5, lays the foundations for the recursive algorithm whose task is establishing whether an instance of SEFE admits a SEFE.

In order to formulate a proper recursive algorithm based on Theorem 6 we need to ensure that the subsequent instances G_U on which we will call the algorithm recursively

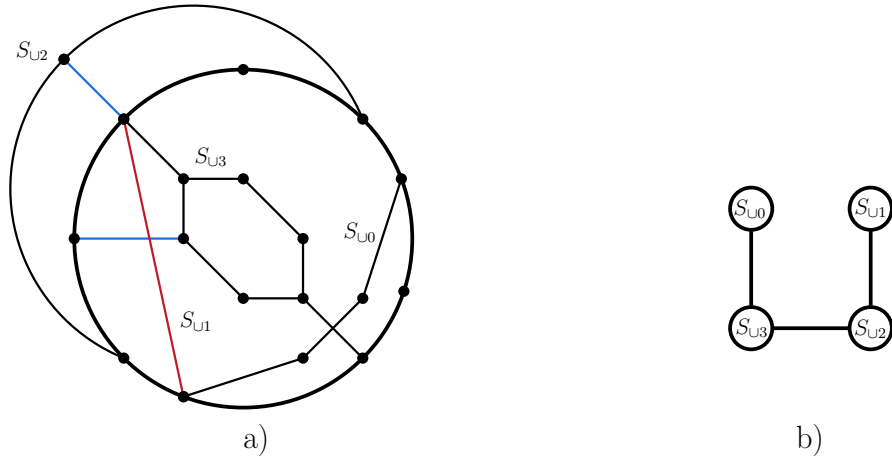


Figure 3.3: Figure a) depicts a Black cycle, and its four union segments, S_{U0} , S_{U1} , S_{U2} and S_{U3} . Figure b) shows the resulting interlacement graph of these segments. Note that such interlacement graph is bipartite.

will always have a cycle in $G_\cap$. If $G_\cap$ is biconnected, Lemma 2 guarantees that the following instances $G'_\cup = C_B \cup S_\cup$ the algorithm will run on will also have their intersection biconnected, meaning they will also keep containing at least one **Black** cycle.

The proof of the following lemma follows the same steps of the one in its counterpart, Lemma 1.

Lemma 5. *Let $G_\cup$ be an instance of SEFE, with $G_\cap$ biconnected. Let C_B be a non-separating **Black** cycle in $G_\cup$, and let $S_\cup$ be its only union segment, which is neither a chord nor a path. If $G_\cup$ admits a SEFE, there is a separating **Black** cycle C'_B consisting of a subpath of C_B plus a **Black** path γ of $S_\cup$ between two **Black** attachments.*

Proof. Let u and v be two **Black** attachments of $S_\cup$ that are consecutive in the circular ordering of C_B , such that there exists a subpath of C_B between u and v that does not contain any other attachment of $S_\cup$. Let us consider such path α , and let β be the subpath of C_B between u and v different from α . Since $G_\cap$ is biconnected, from Lemma 3 we have that there is a **Black** path γ in $S_\cup$ between u and v . Let C'_B be the **Black** cycle obtained from C_B by replacing α with γ . We have that α is a union segment of $G_\cup$ with respect to C'_B . If $S_\cup$ is not a path, let η be an edge of $S_\cup$ not in γ . There is a union segment of C'_B distinct from α containing η . Therefore, if $S_\cup$ is not a path, then C'_B has at least two union segments and is thus a separating **Black** cycle of $G_\cup$; see Fig. 3.4.

In case no such path α between u and v that does not contain any other attachment of $S_\cup$ exists, it follows from the fact that u and v are consecutive that it exists at least a path α' between them not containing any other **Black** attachment, but containing at least a **Red**, **Blue** or **Blue-Red** attachment. Let w be such attachment in α' . Let β' be the subpath of C_B between u and v different from α' , and let γ' be a **Black** path in $S_\cup$ between u and v . The path β' also has to contain at least a **Red**, **Blue** or **Blue-Red** attachment. Let's call such attachment w' . If $G_\cup$ admits a SEFE, $S_\cup$ of C_B must be drawn entirely inside or entirely outside of C_B . Suppose it is drawn inside C_B . Any path between w and w' in $S_\cup$ (which are placed on *opposite* sides of γ'), has to contain at least a node of γ' , otherwise it would result in a crossing with a **Black** in the drawing, since γ' is entirely **Black**. Let's consider the **Black** cycle C'_B obtained from C_B by replacing α' with γ' in C_B . The nodes w and w' necessarily belong to different union segments of

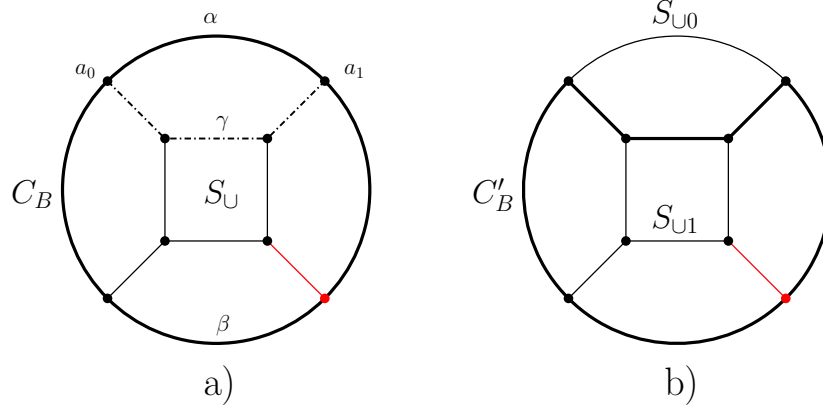


Figure 3.4: In figure a) is depicted a **Black** cycle C_B that does not separate, and its only non-base case union segment S_U . In this case there exist two consecutive **Black** attachment that do not have any other attachment in between, a_0 and a_1 . The path between them α that does not contain other attachment is substituted, in C_B , by the path γ connecting them in S_U . This way α becomes a new union segment S_{U0} of C'_B , and we have another union segment S_{U1} , as shown in figure b).

C'_B , since γ' is contained in C'_B ; see Fig. 3.5. If they belong to the same segment of C'_B , there has to exist a path in S_U connecting them that has no node of γ' . However, if that is the case, such path crosses with γ' , and G_U does not admit a SEFE. $\square$

Lemma 5 shows us that we can always find a separating **Black** cycle in case we have a non-separating one, if the SEFE instance is positive, using the procedure described in

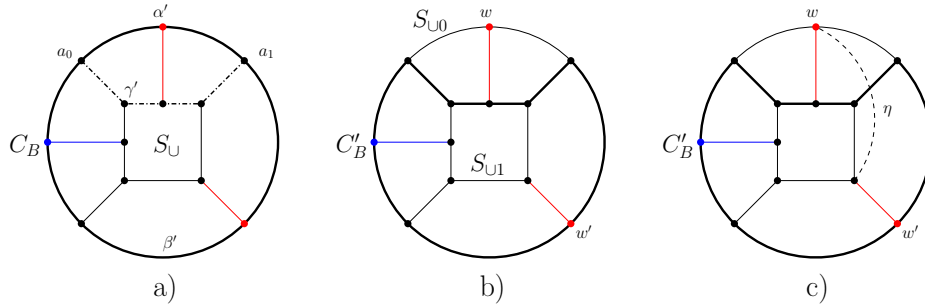


Figure 3.5: Figure a) depicts the case of a **Black** cycle C_B with a non base case union segment S_U such that there are no consecutive **Black** attachments in C_B without other attachments in between. From figure b) we can clearly see that any path in S_U connecting w and w' contains a node of γ' . In figure c) we can see that if this is not the case, there has to exist an edge η (of any color) that crosses with γ' .

the lemma's proof. In case the resulting new Black cycle keeps being non-separating, we can then safely say that the input SEFE instance is negative.

The next concepts in this section are strictly related to computing an actual SEFE of $G_\cup$, assuming it admits one.

Let C_B be a separating Black cycle in $G_\cup$. We have the following definitions, after labeling each node in C_B with a number in $\{0, 1, 2, \dots, k-1\}$ representing the node's encounter index while traversing the cycle.

We define, if such a node exists, the *Red min attachment* in $S_\cup$ of C_B as the Black, Red or Blue-Red attachment of $S_\cup$ with the lowest associated label in C_B . Note that, in general, such a node does not exist if $S_\cup$ is a Blue chord. Likewise, we define, if such a node exists, the *Blue min attachment* in $S_\cup$ of C_B as the Black, Blue or Blue-Red attachment of $S_\cup$ with the lowest associated label in C_B . Note that, in general, such a node does not exist if $S_\cup$ is a Red chord. Specularly, we provide definitions for *max attachments*.

We define, if such a node exists, the *Red max attachment* in $S_\cup$ of C_B as the Black, Red or Blue-Red attachment of $S_\cup$ with the highest associated label in C_B . Likewise, we define, if such a node exists, the *Blue max attachment* in $S_\cup$ of C_B as the Black, Blue or Blue-Red attachment of $S_\cup$ with the highest associated label in C_B .

If a union segment has any *colored min attachment* or *colored max attachment*, we define its *value* as the associated cycle label.

It is worth noting that, given a union segment $S_\cup$ of a Black cycle in $G_\cup$, with $G_\cap$ biconnected, it follows from biconnectivity of $G_\cap$ that if $S_\cup$ has a Red min attachment, it must also have a Red max attachment with a different value, and vice versa (the same argument applies to Blue min attachment and Blue max attachment).

For a union segment $S_\cup$ that has both a *min* and a *max Red attachment*, we also say that it has a *middle Red attachment* if there is at least one Black, Red or Blue-Red between its *min* and a *max Red attachment* (does not matter what attachment it is). Similarly, for a union segment $S_\cup$ that has both a *min* and a *max Blue attachment*, we also say that it has a *middle Blue attachment* if there is at least one Black, Blue or Blue-Red between its *min* and a *max Blue attachment*.

Fig. 3.6 and Fig. 3.7 show examples of this concepts.

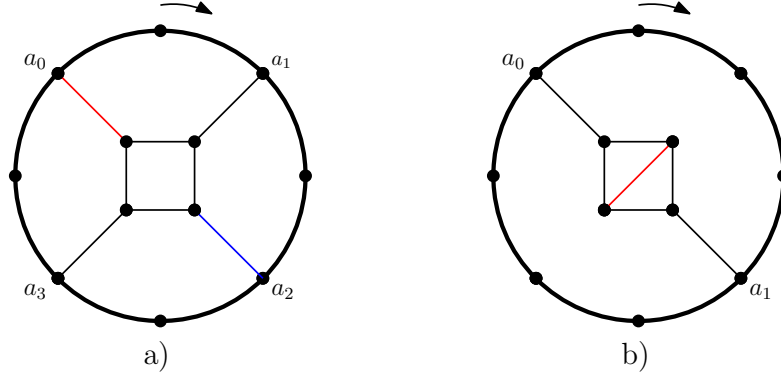


Figure 3.6: In this examples the cycle's starting point, and its traversing order is indicated by the black arrow. The segment's in a) **Red** min attachment is a_1 , while its **Red** max attachment is a_0 , with respective values of 1 and 7. Its **Blue** min attachment is also a_1 , while its **Blue** max attachment is instead a_3 , with respective values of 1 and 5. The segment has both a middle **Red** attachment and a middle **Blue** attachment. The segment's in b) **Red** min attachment is a_1 , as well as its **Blue** min attachment, with a value of 3. Its **Red** max attachment is a_0 , as well as its **Blue** max attachment, with a value of 7. The segment does not have neither a middle **Red** attachment nor a middle **Blue** attachment.

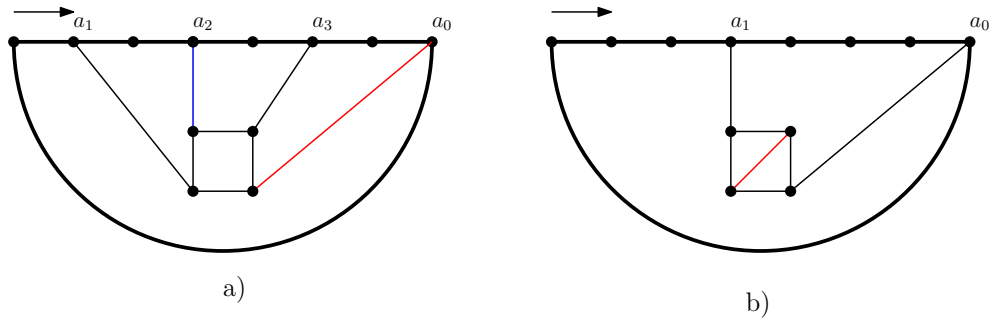


Figure 3.7: This figure represent the same segments from Fig. 3.6 in such a way to make more understandable the concept of min and max attachments, as well as middle attachments. Basically, regarding one of the two projections of the union segment, the min and max attachment of this projection are the most "far away" placed attachments in C_B , the leftmost and the rightmost ones. Regarding the **Blue** portion of the union segment in a), a_2 places itself between its **Blue** min and max attachments, which are respectively a_1 and a_3 . That's why the segment has a **Blue** middle attachment. Same thing applies for its **Red** projection.

3.2 The Testing Algorithm

Let G_U be an instance of SEFE, suppose that it admits a SEFE Γ . If the union graph has a **Black** cycle, then the cycle forms a simple closed curve in Γ ; each of its union segments must be embedded entirely in the interior or exterior region of this curve, otherwise the SEFE would be violated.

In our case study, we investigate a restricted SEFE problem, in particular, we impose $G_\cap$ of G_U to be biconnected. This ensures that there will always be an entirely **Black** cycle in G_U , as discussed in the previous chapter (see Lemma 2).

Theorem 6 and Lemma 2 lay the foundations for the recursive SEFE testing algorithm **TestSEFE**. The function pseudocode is provided in Algorithm 11.

Algorithm 11 Function to check whether an union graphs with biconnected intersection admits a SEFE.

Input: a union graph G_U with $G_\cap$ biconnected, and planar **Red** and **Blue** projections.

Output: *true* if G_U admits a SEFE, *false* otherwise

```

1: procedure TESTSEFE( $G_U$ )
2:    $C_B \leftarrow \text{COMPUTECYCLE}(G_\cap)$  ▷ computes a cycle in  $G_\cap$ 
3:    $segments \leftarrow \text{COMPUTEUNIONSEGMENTS}(G_U, C_B)$ 
4:   if  $|segments| == 1$  then
5:     if  $S_U$  in  $segments$  is a path then ▷ the base case
6:       return true
7:     else ▷ chosen cycle is not separating
8:        $C_B \leftarrow \text{ADJUSTCYCLESEFE}(G_U, S_U, C_B)$ 
9:        $segments \leftarrow \text{COMPUTEUNIONSEGMENTS}(G_U, C_B)$ 
10:      if  $|segments| == 1$  then
11:        return false
12:       $I_{G_U, C_B} \leftarrow \text{COMPUTESEFEINTERLACEMENTGRAPH}(C_B, segments)$ 
13:      if not ISGRAPHBIPARTITE( $I_{G_U, C_B}$ ) then
14:        return false
15:      for  $S_U$  in  $segments$  do
16:         $G' \leftarrow S_U \cup C_B$ 
17:        if not TESTSEFE( $G'$ ) then
18:          return false
19:      return true

```

We can clearly see a lot of similarities between Algorithm 11 and Algorithm 1, since the two algorithms share most of the pseudocode structure. We are now going to dive deep into the sub-algorithms that actually set them apart. The procedures found in the

algorithm, that are not trivial, will have a dedicated section describing them. Function `ComputeUnionSegments` is described in Section 3.2.1, `AdjustCycleSefe` in Section 3.2.2, and `ComputeSefeInterlacementGraph` in Section 2.2.3. Function `ComputeCycle` simply computes a cycle in a graph, while function `IsGraphBipartite` checks if a graph is bipartite. Both these function can easily be implemented with a depth first search or a broad first search.

3.2.1 Computing Union Segments

Computing the union segments of an instance of SEFE G_U given a **Black** cycle C_B , is basically identical to computing segments of a graph G given a cycle C ; what is new is the introduction of colors of edges in the union segment S_U , which we have to keep track of. Also the color of attachment nodes has to be handled, for each union segment S_U . Regarding the color of attachments, note that **Black** edges *take over* other edges color, and an attachment can both be **Red** and **Blue**. Pseudocode for the algorithm can be found in Algorithm 12.

Function `ComputeUnionSegments` utilizes an helper function `DfsSegmentSefe`, whose task it populating a union segment given a node belonging to it, and mark attachments nodes labeling them with their color.

The function's pseudocode is provided in Algorithm 13. It keeps expanding the node until it reaches an already visited node (note that cycle's nodes are already set as

Algorithm 12 Function to compute the union segments of a **Black** cycle in a union graph.

Input: a union graph G_U , a **Black** cycle C_B of G_U

Output: the union segments of C_B in G_U

```

1: procedure COMPUTEUNIONSEGMENTS( $G_U, C_B$ )
2:    $segments \leftarrow$  empty list
3:   for  $v$  in  $C_B$  do
4:     mark  $v$  as visited
5:   for  $v$  in  $V$  do
6:     if  $v$  is not visited then
7:        $S_U \leftarrow$  empty union segment ▷ i.e. empty union graph
8:       DFSSEGMENTSEFE( $G_U, C_B, S_U, v$ )
9:       add  $S_U$  to  $segments$ 
10:  return  $segments$ 
```

Algorithm 13 Helper function for `ComputeUnionSegments` to populate a union segment using a depth first search. It marks attachment nodes of the segment, and label them with their color.

Input: a union graph $G_{\cup}$, a **Black** cycle C_B , a union segment S , a node v

```

1: procedure DFSSEGMENTSEFE( $G_{\cup}, C_B, S_{\cup}, v$ )
2:   add  $v$  to  $S$ 
3:   mark  $v$  as visited
4:   for  $u$  in  $v$  neighbors do
5:      $color \leftarrow (u, v)$  color in  $G_{\cup}$ 
6:     if  $(u, v)$  is not in  $S_{\cup}$  edges then
7:       add  $(u, v)$  to  $S_{\cup}$  edges
8:       label  $(u, v)$  in  $S_{\cup}$  with  $color$ 
9:     if  $u$  is in  $C$  and  $u$  is not in  $S_{\cup}$  nodes then
10:      add  $u$  to  $S_{\cup}$  nodes
11:      mark  $u$  as attachment in  $S_{\cup}$ 
12:      label  $u$  with  $color$  ▷ Black overwrites other colors in  $S_{\cup}$ 
13:      ▷ an attachment in  $S_{\cup}$  can be both Red and Blue
14:     if  $u$  is not visited then
15:       DFSSEGMENT( $G, C, S_{\cup}, u$ )

```

visited). Any edge in the union segment will retain the same color it had in the union graph.

3.2.2 Non-separating Cycles

Non-separating cycles C_B that originate a non-base case union segment $S_{\cup}$ have to be dealt in the SEFE algorithm as well, luckily, the algorithm shares a lot of similarities with Algorithm 4. Lemma 5's proof describes the procedure to compute a separating **Black** cycle from a non-separating one, if the input SEFE instance $G_{\cup}$ is positive. If the resulting new **Black** cycle computed with the procedure described in Lemma 5's proof keeps being non-separating, $G_{\cup}$ does not admit a SEFE. Two consecutive **Black** attachments a_i and a_j have to be found, which will determine the endpoints of the path inside the segment that will substitute a path in the cycle. Note that, from biconnectivity of the union graph's intersection $G_{\cap}$ comes that a non base case segment always has at least two **Black** attachments. From Lemma 3 we know that an entirely **Black** path in $S_{\cup}$ between those two **Black** attachments always exists, which will take place of a path in C_B .

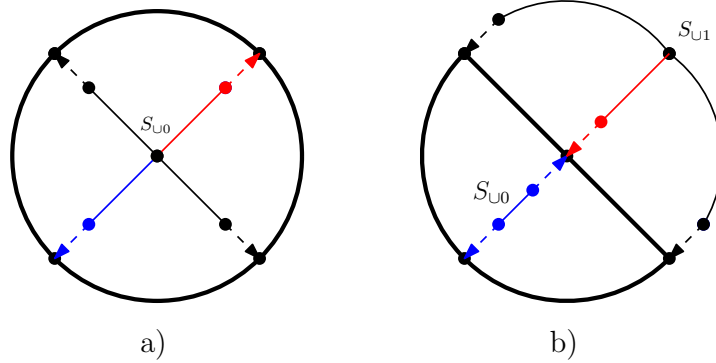


Figure 3.8: Figure a) shows a non-separating cycle with a non-base case union segment S_{U0} . Note that in any path in the cycle between the two black attachments there is another non-Black attachment. Figure b) shows the new separating Black cycle and its union segments S_{U0} (which is a Blue chord) and S_{U1} .

What we need to be aware of is that there is no guarantee that a path in C_B between two Black attachments a_i to a_j in C_B not containing other attachments of S_U exists, since between them there may always be a non-Black attachment. However, when both possible paths in C_B from a_i to a_j contain other attachment nodes of S_U , we can safely choose arbitrarily between any of the two, since if the new found Black cycle is also non-separating, we know from Lemma 5 that G_U does not admit a SEFE..

The algorithm's pseudocode to compute a separating cycle from a non-separating one is described in Algorithm 14.

Firstly, two *consecutive* Black attachments a_0 and a_1 , while traversing the cycle C_B , are found. The helper functions then aid the algorithm to find and change paths using these two attachments. `ComputePathInCycleSefe`'s job is finding a path in C_B connecting a_0 and a_1 that does not contain any other Black attachment of S_U , and its pseudocode is provided in Algorithm 15. `ComputePath` computes a path between two nodes in a graph, assuming it exists. `ChangePath` is instead described in Algorithm 7.

Algorithm 14 Function to compute a separating Black cycle from a non-separating one.

Input: a union segment $S_{\cup}$, the Black cycle C_B

Output: a new *separating* Black cycle C

```

1: procedure ADJUSTCYCLESEFE( $S_{\cup}, C_B$ )
2:    $attachments \leftarrow$  empty list
3:   for  $v$  in  $C_B$  do                                 $\triangleright$  traversing the cycle in clockwise order
4:     if  $v$  is attachment of  $S_{\cup}$  and color of  $v$  is Black then
5:       add  $v$  to  $attachments$ 
6:     if  $|attachments| == 2$  then
7:       break                                           $\triangleright$  list now contains two "consecutive" Black attachments
8:    $n_0 \leftarrow attachments[0]$ 
9:    $n_1 \leftarrow attachments[1]$ 
10:   $cyclePath \leftarrow$  COMPUTEPATHINCYLESEFE( $C_B, n_0, n_1, S_{\cup}$ )
11:   $segmentPath \leftarrow$  COMPUTEPATH( $S_{\cap}, n_0, n_1$ )
12:                                      $\triangleright$  Computes a path between two nodes in a graph
13:  return CHANGEPATH( $C_B, cyclePath, segmentPath$ )     $\triangleright$  Algorithm 7

```

Algorithm 15 uses the already defined function **FollowNode** described in Algorithm 6 to find a path in C_B between the two attachments n_0 and n_1 that does not contain any other attachment n_i of $S_{\cup}$ with $i \neq 0, 1$, regardless of the attachment's color. If no such path exists, any path between n_0 and n_1 that does not contain any Black attachment is good, as we have seen before. In order to find such path, function **FollowNodeSefe** is utilized, which is described in Algorithm 16.

Algorithm 15 Function to compute a path in a Black cycle between two attachments.

Input: a cycle Black C_B , a node n_0 , a node n_1 , a union segment $S_{\cup}$

Output: a list of nodes representing a path

```

1: procedure COMPUTEPATHINCYLESEFE( $C_B, n_0, n_1, S_{\cup}$ )
2:    $u_0 \leftarrow$  first neighbor of  $n_0$  in  $C_B$ 
3:    $u_1 \leftarrow$  second neighbor of  $n_1$  in  $C_B$ 
4:    $path \leftarrow$  FOLLOWNODE( $C_B, n_0, n_1, S_{\cup}, u_0$ )     $\triangleright$  Algorithm 6
5:   if  $path$  is empty then
6:      $path \leftarrow$  FOLLOWNODE( $C_B, n_0, n_1, S_{\cup}, u_1$ )     $\triangleright$  Algorithm 6
7:   if  $path$  is empty then
8:      $path \leftarrow$  FOLLOWNODESEFE( $C_B, n_0, n_1, S_{\cup}, u_0$ )
9:   if  $path$  is empty then
10:     $path \leftarrow$  FOLLOWNODESEFE( $C_B, n_0, n_1, S_{\cup}, u_1$ )
11:  return  $path$ 

```

Algorithm 16 Helper function for `ComputePathInCycleSefe`. The function tries to find a path in the **Black** cycle between two **Black** attachments n_0 and n_1 , following u , one of the two neighbors of n_0 . If following the node u leads to a path with another **Black** attachment, an empty list is otherwise returned.

Input: a **Black** cycle C_B , a node n_0 , a node n_1 , a union segment S_U , a node u

Output: a list of nodes representing a path, or an empty list

```

1: procedure FOLLOWNODESEFE( $C_B, n_0, n_1, S_U, u$ )
2:    $path \leftarrow$  empty list
3:   add  $n_0$  to  $path$ 
4:    $prev \leftarrow n_0$ 
5:   while true do
6:     add  $u$  to  $path$ 
7:     if  $u = n_1$  then
8:       return  $path$ 
9:     if  $u$  is a Black attachment in  $S_U$  then
10:      return empty list  $\triangleright$  We do not want other Black attachments in the path
11:     $temp \leftarrow u$ 
12:    if  $u$ 's first neighbor =  $prev$  then
13:       $u \leftarrow u$ 's second neighbor
14:    else
15:       $u \leftarrow u$ 's first neighbor
16:     $prev \leftarrow temp$ 

```

3.2.3 The SEFE Interlacement Graph

As stated in Theorem 6, in order to check for the instance G_U (with $G_\cap$ biconnected) to admit a SEFE, we need to check whether the SEFE interlacement graph I_{G_U, C_B} built from a separating **Black** cycle and its union segments is bipartite, meaning that the set of the union segments can be partitioned in two sets such that no two union segments from the same set are in conflict. This way one set gets placed outside the **Black** cycle, and the other one inside of it. The pseudocode in Algorithm 17 implements the algorithm to compute I_{G_U, C_B} .

The algorithm uses an helper function, `AreUnionSegmentsInConflict`, to determine whether two union segments are in conflict. This function's pseudocode is provided in Algorithm 18.

When dealing with a Simultaneous Embedding with Fixed Edges of multiple graphs, only intersections between edges belonging to different graphs are allowed. This is clearly equivalent to, given an instance of SEFE G_U , only allow crossings between **Red** and **Blue**

Algorithm 17 Function to compute the Sefe interlacement graph from the Black cycle and the union segments

Input: a Black cycle C_B , the union segments of C_B

Output: the Sefe interlacement graph $I_{G_\cup, C_B}$

```

1: procedure COMPUTESEFEINTERLACEMENTGRAPH( $C_B, segments$ )
2:    $I_{G_\cup, C_B} \leftarrow$  empty graph
3:   for  $S_\cup$  in  $segments$  do
4:     add a node to  $I_{G, C}$ 
5:   for  $0 \leq i < |segments| - 1$  do
6:      $S_{\cup i} \leftarrow segments[i]$ 
7:     for  $i < j < |segments|$  do
8:        $S_{\cup j} \leftarrow segments[j]$ 
9:       if AREUNIONSEGMENTSINCONFLICT( $C_B, S_{\cup i}, S_{\cup j}$ ) then
10:        add  $(i, j)$  to  $I_{G, C}$  edges
11:   return  $I_{G_\cup, C_B}$ 

```

edges in a drawing of $G_\cup$.

We have seen in Section 2.2.3 that conflict between segments can be detected by only analyzing their attachment nodes. We can use this approach also for union segments: we just have to check if any same-colored projection is in conflict. This way we can reuse the already defined concept of conflict between segments for union segments too. Algorithm 18 displays the pseudocode for the function **AreUnionSegmentsInConflict** whose job is determining whether two union segments are in conflict.

Algorithm 18 uses the following helper functions: **RedProjection**, described in Algorithm 19, whose task is projecting an instance of SEFE $G_\cup$ to its **Red** subgraph, **BlueProjection**, described in Algorithm 20, which has the opposite task of projecting an instance of SEFE $G_\cup$ to its **Blue** subgraph, and **AreSegmentsInConflict**, already described in Section 2.2.3, in particular in Algorithm 9.

If the union segment $S_\cup$ given in input is not a **Blue** chord, the projection returned by function **RedProjection** retains all the nodes of $S_\cup$, and all edges of $S_\cup$ whose color is either **Black** or **Red**. The attachments of $S_\cup$ that keep being attachments in the projection are all the attachments labeled as either **Black**, **Red** or **Blue-Red**. If $S_\cup$ is instead a **Blue** chord, an empty segment is returned.

Algorithm 18 Function to check if a pair of union segments is in conflict.

Input: a black cycle C_B , a union segment S_{U0} , a union segment S_{U1}

Output: *true* if the union segments are in conflict, *false* otherwise

```

1: procedure AREUNIONSEGMENTSINCONFLICT( $C_B, S_{U0}, S_{U1}$ )
2:    $red_0 \leftarrow \text{REDPROJECTION}(S_{U0})$ 
3:    $blue_0 \leftarrow \text{BLUEPROJECTION}(S_{U0})$ 
4:    $red_1 \leftarrow \text{REDPROJECTION}(S_{U1})$ 
5:    $blue_1 \leftarrow \text{BLUEPROJECTION}(S_{U1})$ 
6:   if  $red_0$  is not empty and  $red_1$  is not empty then
7:      $\triangleright$  Red projection might be empty if the union segment is a Blue chord
8:     if ARESEGMENTSINCONFLICT( $C_B, red_0, red_1$ ) then  $\triangleright$  Algorithm 9
9:       return true
10:  if  $blue_0$  is not empty and  $blue_1$  is not empty then
11:     $\triangleright$  Blue projection might be empty if the union segment is a Red chord
12:    return ARESEGMENTSINCONFLICT( $C_B, blue_0, blue_1$ )  $\triangleright$  Algorithm 9
13:  return false  $\triangleright$  one segment is a Blue chord and the other is a Red chord

```

Algorithm 20 Function to project a union segment to its Blue component.

Input: a union segment S_U

Output: a segment *blue*

```

1: procedure BLUEPROJECTION( $S_U$ )
2:    $blue \leftarrow$  empty segment
3:   for  $v$  in  $S_U$  nodes do
4:     if  $v$  is an attachment in  $S_U$  and  $v$  attachment color is Red then
5:       continue
6:     add  $v$  to  $blue$  nodes
7:     if  $v$  is an attachment in  $S_U$  then
8:       mark  $v$  as attachment in  $blue$ 
9:   for  $(u, v)$  in  $S_U$  edges do
10:    if  $(u, v)$  color in  $S_U$  is not Red then
11:      add  $(u, v)$  to  $blue$  edges
12:  return  $blue$ 

```

Algorithm 19 Function to project a union segment to its Red component.

Input: a union segment S_U

Output: a segment *red*

```

1: procedure REDPROJECTION( $S_U$ )
2:    $red \leftarrow$  empty segment
3:   for  $v$  in  $S_U$  nodes do
4:     if  $v$  is an attachment in  $S_U$  and  $v$  attachment color is Blue then
5:       continue
6:     add  $v$  to  $red$  nodes
7:     if  $v$  is an attachment in  $S_U$  then
8:       mark  $v$  as attachment in  $red$ 
9:   for  $(u, v)$  in  $S_U$  edges do
10:    if  $(u, v)$  color in  $S_U$  is not Blue then
11:      add  $(u, v)$  to  $red$  edges
12:  return  $red$ 

```

Likewise as function `RedProjection`, if the union segment S_U given in input is not a **Red** chord, the projection returned by function `BlueProjection` retains all the nodes of S_U , and all edges of S_U whose color is either **Black** or **Blue**. The attachments of S_U that keep being attachments in the projection are all the attachments labeled as either **Black**, **Blue** or **Blue-Red**. If S_U is instead a **Red** chord, an empty segment is returned.

It is worth noting that this approach makes sense only because the union graph G_U that originates the segments on which we are checking conflicts, has its intersection $G_\cap$ biconnected, guaranteeing that any projection, if not empty, will remain connected. We could not treat attachments of a projection as attachments of the same segment. Having $G_\cap$ biconnected implies that a union segment is related to, at most, a segment in each of its projection. If that was not the case, we would have to have a slightly more sophisticated checking of conflicts between same colored projections, since if we want to check for conflicts between S_{U_i} and S_{U_j} we would have to check for any connected component in the projection of S_{U_i} to be in conflict with any connected component in the same colored projection of S_{U_j} .

3.3 The Embedding Algorithm

In this section, we will describe procedures to compute, given a positive instance G_U , its simultaneous embedding with fixed edges. The embedding algorithm follows the same intuitions described in Section 3.2.

Function `ComputeSEFE`, whose pseudocode is provided in Algorithm 21, computes the embedding of an instance of SEFE G_U , assuming it admits one. Let C_B be a **Black** cycle in G_U . If C_B only has a union segment that is a base case (i.e. a path or a chord), the simultaneous embedding gets directly computed by the function `EmbedBaseCase`. Otherwise a separating **Black** cycle of G_U is computed. A bipartition of the resulting interlacement graph separates what segments of C_B will be drawn inside the cycle, and what will be drawn outside of it. The cycle will be always assumed to be drawn clockwise. This assumption is necessary since, let S_U be a union segment of C_B and Γ an embedding of $S_U \cup C_B$, Γ by itself does not force any segment to be placed neither inside nor outside the cycle, unless we fix the cycle drawing direction; see Fig. 3.9. In this entire section, we will always keep making this assumption, even when not explicitly

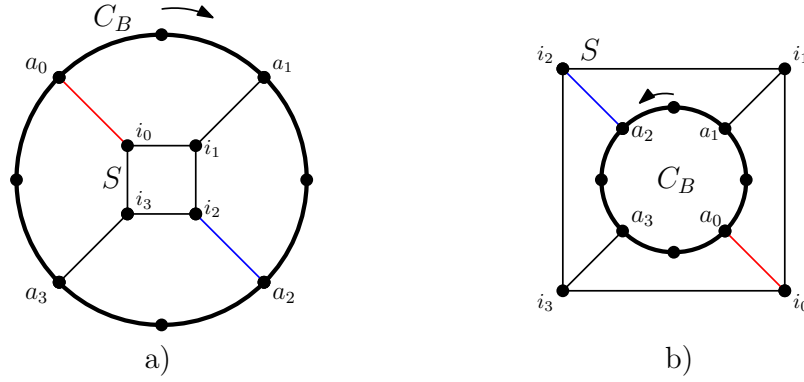


Figure 3.9: The two drawings come from the same simultaneous embedding. We can see in figure a), where the cycle is drawn clockwise, that its union segment is drawn inside the cycle. In figure b) the cycle is instead drawn counter clockwise, and the same union segment is placed outside of the cycle.

affirmed. For each union segment $S_{\cup}$ of the computed cycle C_B , an embedding of $S_{\cup} \cup C_B$ gets recursively computed. Then all these embeddings get merged together to finally build a SEFE of $G_{\cup}$. The procedure utilizes other functions, some of which got already described in Section 3.2, while other are entirely new. New procedures the algorithm makes use of are: **EmbedBaseCase**, described in Algorithm 22; **ComputeBipartition**, which simply computes a bipartition of a bipartite graph G given in input, returning an array associating each node of G with a 0 or a 1; and **MergeSegmentsEmbeddings**, described in Algorithm 23, merges the simultaneous embeddings of the union segments together to obtain the final simultaneous embedding.

Algorithm 21 Function to compute a simultaneous embedding for a positive instance of SEFE $G_{\cup}$ with $G_{\cap}$ biconnected.

Input: a union graph $G_{\cup}$, with $G_{\cap}$ biconnected, that admits a SEFE

Output: the SEFE of $G_{\cup}$

```

1: procedure COMPUTESEFE( $G_{\cup}$ )
2:    $C_B \leftarrow \text{COMPUTECYCLE}(G_{\cap})$  ▷ computes a cycle in  $G_{\cap}$ .
3:    $segments \leftarrow \text{COMPUTEUNIONSEGMENTS}(G_{\cup}, C_B)$  ▷ Algorithm 12.
4:   if  $|segments| == 1$  then
5:     if  $S_{\cup}$  in  $segments$  is a path then ▷ the base case.
6:       return EMBEDBASECASE( $S_{\cup}, C_B$ )
7:     else ▷ chosen cycle is not separating.
8:        $C_B \leftarrow \text{ADJUSTCYCLESEFE}(G_{\cup}, S_{\cup}, C_B)$  ▷ Algorithm 14.
9:        $segments \leftarrow \text{COMPUTEUNIONSEGMENTS}(G_{\cup}, C_B)$  ▷ Algorithm 12.
10:   $I_{G_{\cup}, C_B} \leftarrow \text{COMPUTESEFEINTERLACEMENTGRAPH}(C_B, segments)$ 
11:  ▷ Algorithm 17.
12:   $bip \leftarrow \text{COMPUTE BIPARTITION}(I_{G_{\cup}, C_B})$ 
13:   $embeddings \leftarrow$  empty list
14:  for  $S_{\cup}$  in  $segments$  do
15:     $G'_{\cup} \leftarrow S_{\cup} \cup C_B$ 
16:     $embedding \leftarrow \text{COMPUTESEFE}(G'_{\cup})$ 
17:    add  $embedding$  to  $embeddings$ 
18:  return MERGESEGMENTSEMBEDDINGS( $G_{\cup}, C_B, embeddings, bip, segments$ )

```

Function **EmbedBaseCase**, whose pseudocode is provided in Algorithm 22, has the job of computing, given a base case union segment $S_{\cup}$ and its originating cycle C_B , a simultaneous embedding of $G' = S_{\cup} \cup C_B$. The embedding is computed such that, if the cycle is drawn clockwise, the union segment is placed inside of C_B . Note that, in G' , all the nodes either have 2 neighbors or 3. Clearly, nodes with 3 neighbors are the attachments nodes of $S_{\cup}$, and since $S_{\cup}$ is a base case, only 2 such nodes can exist. The rotation scheme of the nodes in G' with 2 neighbors is arbitrary. The rotation scheme of the nodes in G' with 3 neighbors (i.e. the attachment nodes in $S_{\cup}$) is built using the following rule: let v be such a node, the first and third elements of the rotation scheme of v are, respectively, the next and previous node of v in the cycle C_B (since v has to be an attachment, it also has to belong to C_B), while the second element will be the other neighbor of v .

Algorithm 22 Function to compute a simultaneous embedding for an instance of SEFE $G_U = S_U \cup C_B$, with S_U being a base case segment, i.e. a path or a chord, and C_B being the Black cycle originating it. If C_B is drawn clockwise, S_U gets placed inside of C_B .

Input: a Black cycle C_B and a union segment of his S_U .

Output: the embedding of $G_U = S_U \cup C_B$

```

1: procedure EMBEDBASECASE( $S_U, C_B$ )
2:    $embedding \leftarrow$  empty embedding of  $S_U \cup C_B$ 
3:    $\triangleright$  initializes each node associating it with an empty list.
4:   for  $v$  in  $C_B$  do  $\triangleright$  traversing  $C_B$  in order.
5:     add  $v$ 's next node in  $C_B$  to  $v$ 's adjacency list in  $embedding$ 
6:     if  $v$  is an attachment of  $S_U$  then
7:       add  $v$ 's only neighbor in  $S_U$  to  $v$ 's adjacency list in  $embedding$ 
8:       add  $v$ 's prev node in  $C_B$  to  $v$ 's adjacency list in  $embedding$ 
9:     for  $v$  in  $S_U$  do
10:      if  $v$  is an attachment of  $S_U$  then
11:        continue  $\triangleright$  his neighbors have been already added to  $embedding$ 
12:      add  $v$ 's first neighbor in  $S_U$  to  $v$ 's adjacency list in  $embedding$ 
13:      add  $v$ 's second neighbor in  $S_U$  to  $v$ 's adjacency list in  $embedding$ 
14:   return  $embedding$ 
```

Let C_B be a separating cycle of a positive instance of SEFE G_U .

Function **MergeSegmentsEmbeddings**, whose pseudocode is described in Algorithm 23, merges together the simultaneous embeddings of all the union segments of C_B with C_B , returning, that way, a simultaneous embedding of G_U . Given a bipartition of the interlacement graphs of the union segments, it iterates on all the nodes on the cycle and, for each node v , it computes an ordering of the incident union segments to the node. This ordering gets computed by function **ComputeOrder** for segments placed inside the cycle and outside the cycle separately. The ordering is computed, assuming the segments need to be placed outside the cycle (and that is why the order of the segments inside the cycle gets reversed right after getting computed), such that it identifies the around ordering of segments encountering while traversing the neighbors of the node v clockwise; see Fig. 3.10. Remember that we are always assuming the cycle is drawn clockwise in the final embedding. However, for each union segment S_U , the simultaneous embedding given of $S_U \cup C_B$, that got computed recursively, may either have the segment S_U placed inside or outside of the clockwise drawn cycle. If the simultaneous embedding of $S_U \cup C_B$ places S_U inside of C_B , but we need to place it outside (or vice versa), we

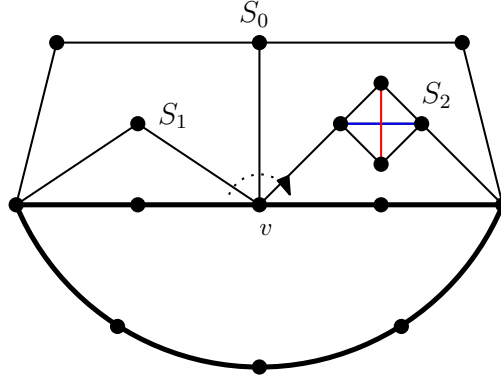


Figure 3.10: This figure shows the union segments adjacent to a cycle node v . In this case, the clockwise encountering order of segments, regarding v , is S_1 , S_0 and then S_2 .

need to reverse the rotation scheme of all its nodes, in order to make it "*compatible*". This incompatibilities are detected with the help of function `IsSegmentInsideCycle`, whose pseudocode is in Algorithm 24. It populates the rotation scheme of v in the final simultaneous embedding by firstly adding v 's next node in C_B , then the edges of union segments that have to be placed inside of C_B , then it adds v 's previous node in C_B , and lastly the edges of union segments that have to be placed outside of C_B . Function `AddMiddleEdges` is used for these last steps. The procedure then finally finishes its task by adding the rotation schemes of internal nodes of each segment, to the final simultaneous embedding, using function `AddInternalNodesEdges`.

Algorithm 23 Function to compute SEFE for an instance of SEFE $G_{\cup}$ given a Black cycle C_B of $G_{\cup}$, and a list of SEFEs Γ_i containing a SEFE of $S_{\cup i} \cup C_B$ for each union segment of C_B .

Input: an instance of SEFE $G_{\cup}$, a Black cycle C_B , a list of SEFEs, bip , a bipartition of the interlacement graph of C_B 's segments, and the list of segments.

Output: the SEFE of $G_{\cup}$

```

1: procedure MERGESEGMENTSEMBEDDINGS( $G_{\cup}, C_B, embeddings, bip, segments$ )
2:    $output \leftarrow$  empty embedding for  $G_{\cup}$ 
3:   for  $0 \leq i < |segments|$  do
4:      $isInside \leftarrow$  ISSEGMENTINSIDECYCLE( $C_B, segments[i], embeddings[i]$ )
5:     if ( $bip[i] = 0$  xor  $isInside$ ) then
6:       reverse rotation scheme of  $embeddings[i]$ 
7:   for  $v$  in  $C_B$  do  $\triangleright$  traversing  $C_B$  in order.
8:      $insideSegments \leftarrow$  empty list
9:      $outsideSegments \leftarrow$  empty list
10:    for  $0 \leq i < |segments|$  do
11:      if  $v$  is not in  $segments[i]$  nodes then continue
12:      if  $bipartition[i] = 0$  then add  $i$  to  $insideSegments$ 
13:      else add  $i$  to  $outsideSegments$ 
14:     $insideOrder \leftarrow$  COMPUTEORDER( $v, segments, insideSegments$ )
15:    reverse  $insideOrder$ 
16:     $outsideOrder \leftarrow$  COMPUTEORDER( $v, segments, outsideSegments$ )
17:    add  $v$ 's next node in  $C_B$  to  $v$ 's adjacency list in  $embedding$ 
18:    for  $0 \leq i < |insideOrder|$  do
19:       $S_{\cup} \leftarrow segments[insideOrder[i]]$ 
20:       $\Gamma \leftarrow embeddings[insideOrder[i]]$ 
21:      ADDMIDDLEEDGES( $C_B, S_{\cup}, true, \Gamma, output$ )
22:    add  $v$ 's previous node in  $C_B$  to  $v$ 's adjacency list in  $embedding$ 
23:    for  $0 \leq i < |outsideOrder|$  do
24:       $S_{\cup} \leftarrow segments[outsideOrder[i]]$ 
25:       $\Gamma \leftarrow embeddings[outsideOrder[i]]$ 
26:      ADDMIDDLEEDGES( $C_B, S_{\cup}, false, \Gamma, output$ )
27:  for  $0 \leq i < |segments|$  do
28:    AddInternalNodesEdges()
29:  return  $output$ 

```

Let Γ be a simultaneous embedding of $G' = S_{\cup} \cup C_B$, where $S_{\cup}$ is a union segment of C_B . Function `IsSegmentInsideCycle`, whose pseudocode is given in Algorithm 24, checks whether drawing C_B clockwise, results in the drawing of Γ placing $S_{\cup}$ inside of C_B . This can be verified by checking if, given an attachment node v of $S_{\cup}$, in the rotation

scheme of v in Γ there is any edge between u and w , where u and w are, respectively, the next and previous nodes of v in the cycle C_B .

Algorithm 24 Given a cycle, assuming it is drawn clockwise, this function determines if a segment of the cycle gets placed inside the cycle, given the embedding of their union.

Input: a Black cycle C_B , a union segment S_U , an embedding Γ of $C_B \cup S_U$

Output: true if drawing the cycle C_B clockwise places the segment S_U inside of C_B .
false otherwise.

```

1: procedure ISSEGMENTINSIDECYCLE( $C_B, S_U, \Gamma$ )
2:    $a \leftarrow$  attachment of  $S_U$  ▷ any attachment is good.
3:    $prev \leftarrow$  node before  $a$  in  $C_B$ 
4:    $next \leftarrow$  node after  $a$  in  $C_B$ 
5:    $list \leftarrow$  adjacency list of  $a$  in  $\Gamma$ 
6:    $pos \leftarrow$  position of  $next$  in  $list$ 
7:    $pos' \leftarrow (pos + 1) \bmod |list|$ 
8:   if  $list[pos'] = prev$  then
9:     return false
10:  else
11:    return true

```

Function **AddMiddleEdges**, described in Algorithm 25, has the job of adding edges of Black cycle C_B node that belong to a union segment S_U in the output simultaneous embedding. It follows the rotation scheme of the edges found in the embedding of $G'_U = C_B \cup S_U$.

Function **AddInternalNodesEdges**, described in Algorithm 26 adds all the edges of non-attachment nodes of a union segment, to the output simultaneous embedding. It follows the rotation scheme of the embedding Γ of $G'_U = C_B \cup S_U$.

Let v be a node in C_B . Let $S_{U0}, S_{U1}, \dots, S_{Uk}$ be a set of union segments that have v as an attachment node. Let pos indicate the index position of v in C_B . Function **ComputeOrder**, described in Algorithm 27, computes a permutation of the segments clockwise order around v such that they do not intersect with each other. In order to do so, the functions **CompareSegments** and **HandleSegmentsDraw** are used. **CompareSegments** has the job of establishing which of the two segments given in input has to be placed before the other, and it gets called on all possible pairs of segments. It takes a color as input, meaning it tries to figure an ordering of the two segments, indirectly relying on their projection. If their Red projection induces an ordering, such ordering is used. If looking for a Red ordering results in a draw, it looks for an ordering using their Blue projections.

Algorithm 25 Function that add to the output embedding the edges of a Black cycle node, that belongs to a union segment.

Input: a Black cycle C_B , a union segment S_U , a boolean value $isIndide$, an embedding Γ of $C_B \cup S_U$, the final embedding *output*

```

1: procedure ADDMIDDLEEDGES( $C_B, S_U, isIndide, \Gamma, output$ )
2:   if  $isInside$  then
3:      $nodeToFind \leftarrow$  next node of  $v$  in  $C_B$ 
4:   else
5:      $nodeToFind \leftarrow$  previous node of  $v$  in  $C_B$ 
6:    $pos \leftarrow$  position of  $nodeToFind$  in  $v$ 's rotation scheme in  $\Gamma$ 
7:   for  $1 \leq i < |v\text{'s neighbors}|$  do
8:      $index \leftarrow (i + pos) \bmod |v\text{'s neighbors}|$ 
9:      $next \leftarrow (v\text{'s rotation scheme in } \Gamma)[index]$ 
10:    if  $next$  is in  $C_B$  then
11:      continue
12:    add  $next$  to  $v$ 's rotation scheme in output

```

Algorithm 26 Function that add to the output embedding the edges of non-attachment nodes that belongs to a union segment.

Input: a Black cycle C_B , a union segment S_U , an embedding Γ of $C_B \cup S_U$, the final embedding *output*

```

1: procedure ADDINTERNALNODESEGES( $C_B, S_U, \Gamma, output$ )
2:   for  $v$  in  $S_U$  do
3:     if  $v$  is in  $C_B$  then
4:       continue
5:     for  $0 \leq i < |v\text{'s neighbors}|$  do
6:        $next \leftarrow (v\text{'s rotation scheme in } \Gamma)[i]$ 
7:       add  $next$  to  $v$ 's rotation scheme in output

```

If looking for a Blue ordering also results in a draw, function `HandleSegmentsDraw` is called.

Algorithm 27 Assumes the cycle is drawn clockwise. Given the position pos of a node in the cycle, this function determines the clockwise order of segments, that share the cycle node in pos , assuming they all need to be placed outside of the cycle, such that they do not create intersections.

Input: a Black cycle C_B ,

Output: true

```

1: procedure COMPUTEORDER( $pos, segments, incidentSegmentsIndexes$ )
2:    $order \leftarrow$  copy of  $incidentSegmentsIndexes$ 
3:   for  $0 \leq i < |order| - 1$  do
4:      $min \leftarrow i$ 
5:      $minSeg \leftarrow segments[order[min]]$ 
6:     for  $i < j < |order|$  do
7:        $newSeg \leftarrow segments[order[j]]$ 
8:        $c \leftarrow \text{COMPARESEGMENTS}(pos, minSeg, newSeg, \text{Red})$ 
9:       if  $c = 0$  then
10:         $c \leftarrow \text{COMPARESEGMENTS}(pos, minSeg, newSeg, \text{Blue})$ 
11:       if  $c < 0$  then
12:         continue
13:       if  $c > 0$  then
14:          $min \leftarrow j$ 
15:          $minSeg \leftarrow segments[order[min]]$ 
16:       continue
17:    $i_0 \leftarrow order[min]$ 
18:    $i_1 \leftarrow order[j]$ 
19:   if  $\text{HANDLESEGMENTS}(pos, minSeg, i_0, newSeg, i_1)$  then
20:      $min \leftarrow j$ 
21:      $minSeg \leftarrow segments[order[min]]$ 
22:   swap  $order[min]$  with  $order[i]$ 
23:   return order

```

Let $S_{\cup 0}$ and $S_{\cup 1}$ be two union segments of a Black cycle C_B , with both segments sharing an attachment node v , identified by its position pos in C_B . Assuming these union segments both have to be placed outside of C_B , which is considered to be drawn clockwise, function **CompareSegments**, whose pseudocode is provided in Algorithm 28, given a **Color** between **Red** and **Blue** in input, has the job of establishing whether the projections of the two union segment of color **Color** induce an ordering of placement of the union segments around v . They induce an ordering if, in order to avoid crossings in the drawing, we have to necessarily place a segment before the other. This ordering takes as reference the cycle node v , and starts placing the segments around v in clockwise

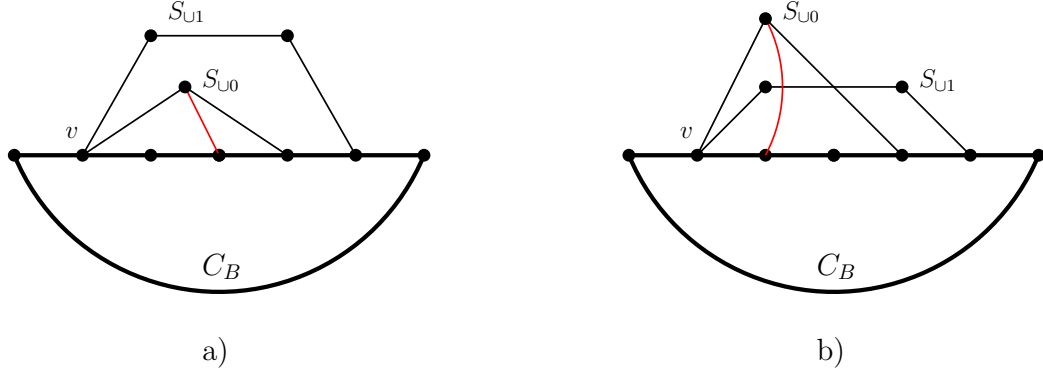


Figure 3.11: In this SEFE instance, with reference to the node v , belonging to both segments S_{U0} and S_{U1} , we must place the segments in a specific clockwise order around v in order to avoid crossings. Specifically, we need to place S_{U0} after S_{U1} , as shown in figure a). If we do not follow this ordering, such as in figure b), there will be crossings.

direction, outside of C_B ; see Fig. 3.11

If the first segment has to be placed before the second, the function returns -1 . If the first has to be placed after the second, the function returns 1 . If both orderings do not result in any crossing at all, 0 is returned.

Inside this function the concepts of **Red/Blue** min/max attachments seen at the end of Section 3.1 finally come to play, as well as **Red/Blue** middle attachments.

Let's now understand how the function determines the ordering, given the projection color **Color**. We will assume the reference color is **Red**, without loss of generality. We will analyze each possibility case by case. In case at least one of the two union segment is a **Blue** chord, no ordering can be established. In case v is the **Red** min attachment of both union segments:

- the union segment with the highest **Red** max attachment needs to be placed before the other (Fig. 3.11 shows this exact case scenario),
- if they also have the same **Red** max attachment, and a union segment has a **Red** middle attachment, this union segment needs to be placed after the other (note that they cannot both have a **Red** middle attachment without compromising the SEFE); see Fig. 3.12 a),

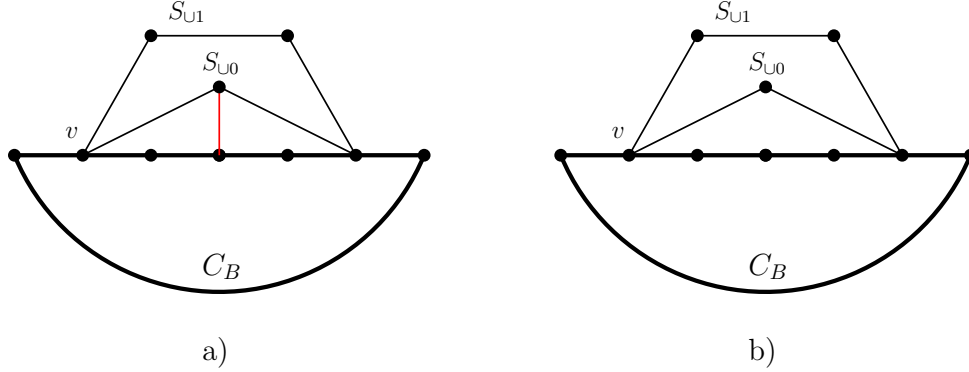


Figure 3.12: The union segments in both figures share the same **Red** min and max attachments. In a) S_{U0} has a **Red** middle attachment, while in b) S_{U0} does not. Note that in figure a), the **Blue** projection of the union segments does not induce an ordering, since S_{U0} does not have a **Blue** middle attachment, it results in a draw, like the one depicted in b).

- if they have the same **Red** max attachment, and no segment has a **Red** middle attachment, the result is a draw; see Fig. 3.12 b).

In case v is the **Red** min attachment of only one union segment, for example S_{U0} , then v has to either be a **Red** max attachment or a **Red** middle attachment of S_{U1} . In both this cases, shown respectively in Fig. 3.13 a) and b), S_{U1} has to be placed before S_{U0} .

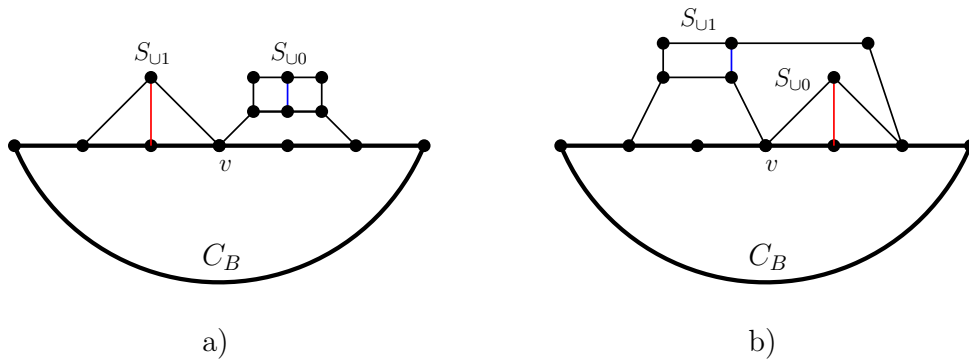


Figure 3.13: In this SEFE instance, with reference to the node v , belonging to both segments S_{U0} and S_{U1} , we must place S_{U0} after S_{U1} in both figures a) and b) in order to avoid crossings. In figure a) v is a **Red** min attachment of S_{U0} and a **Red** max attachment of S_{U1} . In figure a) v keeps being a **Red** min attachment of S_{U0} and it is a **Red** middle attachment of S_{U1} .

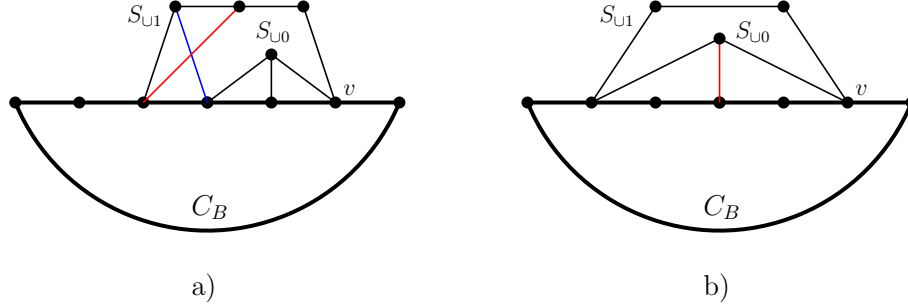


Figure 3.14: In this SEFE instance, with reference to the node v , which is a **Red** max attachment of both union segments S_{U0} and S_{U1} in both figures a) and b), we must place S_{U0} before S_{U1} in both figures a) and b) in order to avoid crossings. In figure a) S_{U0} has a **Red** min attachment with a lower value of the one of S_{U1} . In figure b) S_{U0} has a **Red** middle attachment.

We have similar (opposite) rules in case v is a **Red** max attachment of both union segments:

- the union segment with the lowest **Red** min attachment needs to be placed after the other, as shown in (Fig. 3.14 a),
- if they also have the same **Red** min attachment, and a union segment has a **Red** middle attachment, this union segment needs to be placed before the other; see Fig. 3.14 b),
- if they have the same **Red** min attachment, and no segment has a **Red** middle attachment, the result is a draw, as shown in the previous figure Fig. 3.12 b), but with v being the right attachment instead of the left one.

Likewise, if v is the **Red** max attachment of only one union segment, for example S_{U0} , then v has to either be a **Red** min attachment or a **Red** middle attachment of S_{U1} . In both this cases, shown respectively in Fig. 3.15 a) and b), S_{U1} has to be placed after S_{U0} .

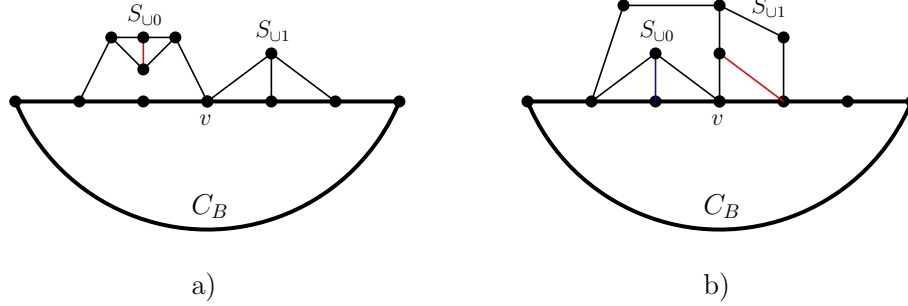


Figure 3.15: In this SEFE instance, with reference to the node v , which is a Red max attachment of S_{U0} in both figures a) and b), we must place S_{U0} before S_{U1} in both figures a) and b) in order to avoid crossings. In figure a) v is a Red min attachment of S_{U1} , while in figure b) it is a Red middle attachment.

Function `HandleSegmentsDraw` has the job of establishing an ordering between union segments S_{U0} and S_{U1} for which any ordering is admissible. This happens when both segments share the same two attachments a_0 and a_1 . In this case, what is important is just keeping the ordering consistent such they do not intersect with each other. If S_{U0} comes before S_{U1} in attachment a_0 's rotation scheme, then S_{U0} has to be placed after S_{U1} in a_1 's rotation scheme, and vice versa; see Fig. 3.16.

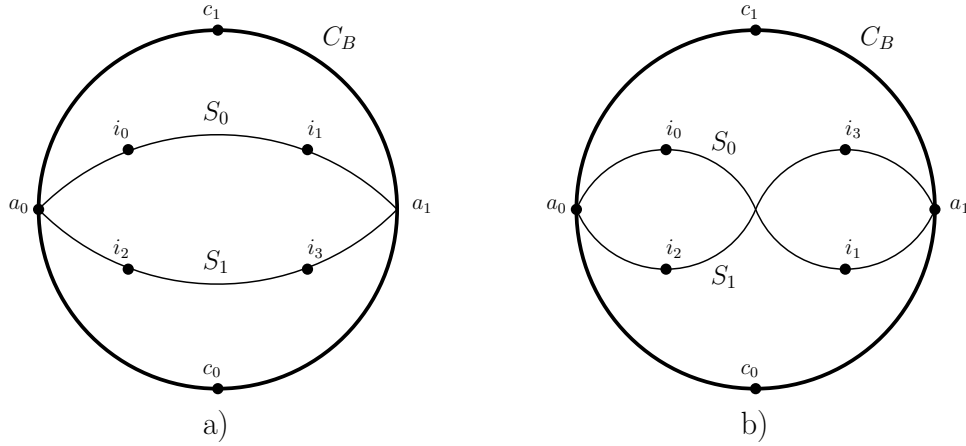


Figure 3.16: Figure a) depicts orderings of the two attachments such that the union segments do not cross with each other, while figure b) represents the opposite case in which they do.

Algorithm 28 Given two union segments of a Black cycle, both sharing an attachment v , this function returns an index which establishes, assuming these segments need to be placed outside the cycle, the clockwise order around v that they have to be placed in, such that crossings are avoided.

Input: the position pos of a node in the Black cycle, a union segment S_{U0} , a union segment S_{U1} , a color **Color**

Output: -1 if the first segment has to be placed before the second; 1 if the first has to be placed after the second; 0 if both orderings do not result in any crossing.

```

1: procedure COMPARESEGMENTS( $pos, S_{U0}, S_{U1}, \text{Color}$ )
2:   if  $\text{Color} = \text{Red}$  then
3:      $\text{Opposite} \leftarrow \text{Blue}$ 
4:   else
5:      $\text{Opposite} \leftarrow \text{Red}$ 
6:   if  $S_{U0}$  is a chord of color  $\text{Opposite}$  then
7:     return  $0$ 
8:   if  $S_{U1}$  is a chord of color  $\text{Opposite}$  then
9:     return  $0$ 
10:   $S_{U0min} \leftarrow \text{Color min attachment of } S_{U0}$ 
11:   $S_{U0max} \leftarrow \text{Color max attachment of } S_{U0}$ 
12:   $S_{U1min} \leftarrow \text{Color min attachment of } S_{U1}$ 
13:   $S_{U1max} \leftarrow \text{Color max attachment of } S_{U1}$ 
14:  if  $pos = S_{U0min}$  and  $pos == S_{U1min}$  then
15:    if  $(S_{U0max} < S_{U1max})$  then return  $1$ 
16:    if  $(S_{U0max} > S_{U1max})$  then return  $-1$ 
17:    if  $S_{U0}$  has  $\text{Color}$  middle attachment then return  $1$ 
18:    if  $S_{U1}$  has  $\text{Color}$  middle attachment then return  $-1$ 
19:    return  $0$ 
20:  if  $pos = S_{U0min}$  or  $pos = S_{U1min}$  then
21:    if  $(S_{U0min} < S_{U1min})$  then return  $-1$ 
22:    if  $(S_{U0min} \geq S_{U1min})$  then return  $1$ 
23:  if  $pos = S_{U0max}$  and  $pos == S_{U1max}$  then
24:    if  $(S_{U0min} < S_{U1min})$  then return  $1$ 
25:    if  $(S_{U0min} > S_{U1min})$  then return  $-1$ 
26:    if  $S_{U0}$  has  $\text{Color}$  middle attachment then return  $-1$ 
27:    if  $S_{U1}$  has  $\text{Color}$  middle attachment then return  $1$ 
28:    return  $0$ 
29:  if  $pos = S_{U0max}$  or  $pos = S_{U1max}$  then
30:    if  $(S_{U0max} < S_{U1max})$  then return  $-1$ 
31:    if  $(S_{U0max} \geq S_{U1max})$  then return  $1$ 
32:  return  $0$ 

```

Algorithm 29 Given two union segments that share the same two attachments, this function establishes an ordering between such segments around the cycle node in position pos .

Input: the position of a node pos , a union segment S_{U0} , a union segment S_{U1} , an index $S_{U0}index$, an index $S_{U1}index$

Output: true if the ordering between the two segments needs to be swapped, false otherwise

```

1: procedure HANDLESEGMENTSDRAW( $pos, S_{U0}, S_{U1}, S_{U0}index, S_{U1}index$ )
2:    $isCycleNodeMinAttachment \leftarrow \text{false}$ 
3:   if  $S_{U0}$ 's Red min attachment =  $pos$  then
4:      $isCycleNodeMinAttachment \leftarrow \text{true}$ 
5:   if  $S_{U1}$ 's Red min attachment =  $pos$  then
6:      $isCycleNodeMinAttachment \leftarrow \text{true}$ 
7:   if  $S_{U0}$ 's Blue min attachment =  $pos$  then
8:      $isCycleNodeMinAttachment \leftarrow \text{true}$ 
9:   if  $S_{U1}$ 's Blue min attachment =  $pos$  then
10:     $isCycleNodeMinAttachment \leftarrow \text{true}$ 
11:    $change \leftarrow (S_{U0}index < S_{U1}index)$ 
12:   if  $isCycleNodeMinAttachment$  then  $change \leftarrow \text{not } change$ 
13:   return  $change$ 

```

Note that in Algorithm 29's body the ordering of the union segments is based on their indexes. This is a simple neat trick to keep such ordering consistent. Also, its gets reversed in case the position pos is the position of a min attachment of the two union segment. This way, considering the two shared attachments a_0 and a_1 of the union segments, the ordering of a_0 is the opposite of the ordering of a_1 .

Chapter 4

The Web Tool

One of the objectives of this work, as well as defining a correct algorithm capable of deciding if two graphs with biconnected intersection admit a simultaneous embedding with fixed edges, was implementing the after mentioned algorithm using a low level programming language, in order for it to be as efficient as possible. Moreover, a concern of ours was let everyone have access to an easy to use site, providing APIs to the SEFE implemented algorithms. In particular, we wanted to offer access to the `Embedder` and `EmbedderSefe` classes of our library, which both have a public function `EmbedGraph` that returns an `optional` that contains, in case of a positive input instance, the planar embedding or the SEFE.

4.1 WebAssembly

Since accessibility trough Web is a priority of ours, but so is efficiency, a clear solution showed up: `WebAssembly`. It allowed us to write an implementation of our algorithm using a low level performing programming language: `C++`.

`WebAssembly` is a low-level, assembly-like language featuring a compact binary format that delivers near-native performance. It can be compiled from various languages, including `C/C++` and `Rust`. Designed to work alongside `JavaScript`, `WebAssembly` allows both languages to function together seamlessly. It is made with portability and security at its core, operating within a sandboxed environment.

The `WebAssembly` code, or bytecode, is optimized for execution on a portable virtual

stack machine. This bytecode is faster to parse and execute than JavaScript and has a more compact representation.

4.2 Description of the Web Tool

The Web Tool’s user interface is made with a plain HTML, CSS and JavaScript combination. The algorithms at its core, however, are built in C++, and later compiled into WebAssembly using the Emscripten compiler. In order to visualize the results of the embedding processes, we opted for a C++ library: OGDF [CGJ⁺23]. After compiling it into WebAssembly, it is used to produce SVG files to visualize the final results.

The tool offers the ability to compute the planar embedding of a given input graph using the Auslander and Parter algorithm, as well as the ability to compute the Simultaneous Embedding with Fixed Edges of two graphs with biconnected intersection. In both cases, if the instances are positive, a visualization of the aforementioned planar embeddings is displayed on the screen. In the case of SEFE, however, for the sake of simplicity, instead of displaying the actual simultaneous embedding, two separate visualizations are shown, one for each input graph. The tool also offers a link to example graphs, some of which are positive instances, while some are negative, and it also provides a link to this thesis PDF.

4.3 The Web Tool Interface

The page responsible for checking and embedding graphs is shown in Fig. 4.1. We can see that, after giving a properly formatted graph in input as a .txt file, it computes, in case the graph is planar, a planar drawing built using the Auslander and Parter algorithm implemented in our library. The tool allows to download the embedding, also using a .txt format, and the .svg file containing the drawing shown at screen.

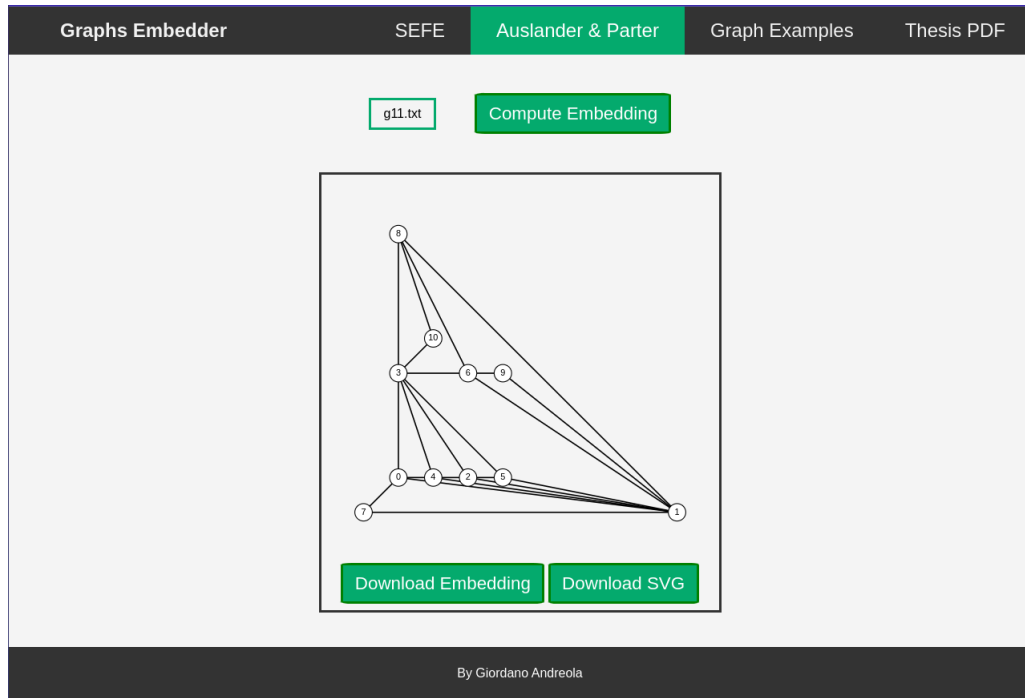


Figure 4.1: The figure shows a screenshot of the Auslander and Parter page in our web tool.

The page responsible for checking, and embedding, two graphs simultaneously is instead shown in Fig. 4.2. After giving the representation of the two graph, which must have a biconnected intersection, we can try to compute the SEFE and in case the instance is positive, a representation is shown at screen. The user may download the SEFE, the induced embedding of the first graph, and the induced embedding of the second graph. It is also possible to download the two `.svg` files representing the two drawings.

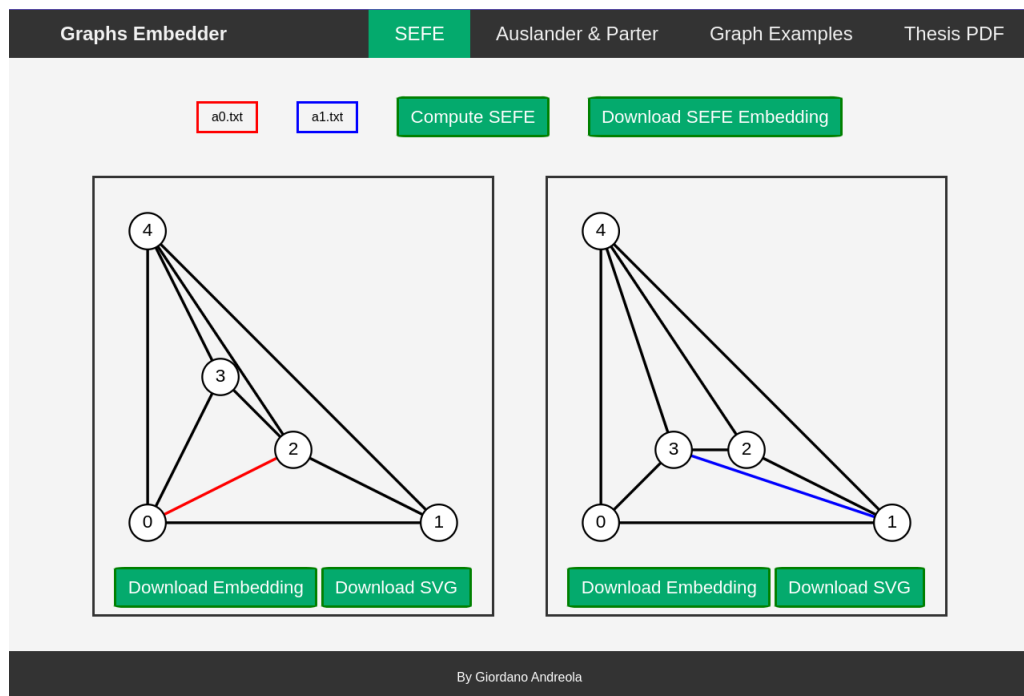


Figure 4.2: The figure shows a screenshot of the SEFE page in our web tool.

Conclusions and Future Developments

In this thesis, we had the goal of defining a novel, simple recursive algorithm capable of testing, with polynomial running time, if two input graphs with a biconnected intersection admit a SEFE, and in case they do, provide such a SEFE. The algorithm we propose retains the time complexity of the algorithm devised by Auslander and Parter, which is $O(|V|^2)$, where V is the set of vertices of the input graphs.

In order to lay the foundations for the design of our algorithm for the SEFE problem of instances with a biconnected intersection, the starting point of our investigation was the analysis of the recursive planarity testing algorithm by Auslander and Parter and the engineering of this algorithm as a software library. After these objectives were achieved, we were then able to develop a recursive algorithm for testing SEFE for the considered instances. The algorithm is conceptually close to the Auslander and Parter’s planarity testing algorithm, and its simplicity allowed it to be smoothly implementable.

Both implementations were made using a low level programming language, `C++`, and the visualizations of the embeddings were made possible by the use of the `OGDF` library.

Our last goal was providing easy public access to the implemented algorithms, and in order to achieve such an objective, the choice fell on compiling the code into `WebAssembly`, using the Emscripten compiler. In this way, we were able to develop a web tool that allows users to access our APIs through a convenient web page.

Regarding the future developments of this work, since our techniques extends to the case of $k \geq 2$ input graphs, as long as their pairwise intersection is the same (the sunflower SEFE restriction), and it is biconnected, a future objective is, indeed, extending

the algorithm to handle more than two graphs.

Another intriguing development would be to provide an actual simultaneous visualization of the computed SEFE. At the moment, for simplicity, we draw the simultaneous embedding as two separate drawings depicting the two graphs, however a better choice would be visualizing them in a unique representation where edges belonging to both graphs are, for example, drawn in black, while edges exclusively belonging to one of the two graphs are drawn in red and blue, depending on the graph they belong to.

Bibliography

- [ADF⁺12] Patrizio Angelini, Giuseppe Di Battista, Fabrizio Frati, Maurizio Patrignani, and Ignaz Rutter. Testing the simultaneous embeddability of two graphs whose intersection is a biconnected or a connected graph. *Journal of Discrete Algorithms*, 14:150–172, 2012. Selected papers from the 21st International Workshop on Combinatorial Algorithms (IWOCA 2010).
- [ADN15] Patrizio Angelini, Giordano Da Lozzo, and Daniel Neuwirth. Advancements on safe and partitioned book embedding problems. *Theoretical Computer Science*, 575:71–89, 2015. Special Issue on Algorithms and Computation.
- [AP61] L. Auslander and S. V. Parter. On imbedding graphs in the sphere. *Journal of Mathematics and Mechanics*, 10(3):517–523, 1961.
- [BETT99] Giuseppe Di Battista, Peter Eades, Roberto Tamassia, and Ioannis G. Tollis. *Graph Drawing: Algorithms for the Visualization of Graphs*. Prentice Hall, Upper Saddle River, NJ, 1999.
- [BKR15] Thomas Bläsius, Annette Karrer, and Ignaz Rutter. Simultaneous embedding: Edge orderings, relative positions, cutvertices, 2015.
- [CGJ⁺23] M. Chimani, C. Gutwenger, M. Jünger, G. W. Klau, K. Klein, and P. Mutzel. The open graph drawing framework (ogdf), 2023.
- [Gol63] AJ Goldstein. An efficient and constructive algorithm for testing whether a graph can be embedded in a plane. In *Graph and Combinatorics Conference*, pages 16–18, 1963.

-
- [HT73] John Hopcroft and Robert Tarjan. Algorithm 447: efficient algorithms for graph manipulation. *Commun. ACM*, 16(6):372–378, jun 1973.
- [Jor87] Camille Jordan. *Cours d'Analyse de l' Ecole Polytechnique*, pages 587–594. Gauthier-Villars, 1887.
- [JS09] Michael Jünger and Michael Schulz. Intersection graphs in simultaneous embedding with fixed edges. *Journal of Graph Algorithms and Applications*, 13(2):205–218, Feb. 2009.
- [Zei] S. Zeijlemaker. Planarity testing: Implementation and analysis of the auslander-partner algorithm. Bachelor's thesis, Eindhoven University of Technology, 2018.